

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
— o0o —



Báo cáo Đồ án II

**Các giải thuật tìm kiếm cục bộ cho bài toán định tuyến cung
bằng drone nhiều chuyển với hàm mục tiêu Min-Max**

Giáo viên hướng dẫn:

Sinh viên thực hiện:

Họ và tên	MSSV
Phạm Anh Khôi	20230043

Hà Nội - 2026

Mục lục

1	Tổng quan bài toán và các công trình liên quan	5
1.1	Dặt vấn đề	5
1.2	Mô tả khái quát bài toán MM-MT-dLARP	6
1.3	Tổng quan về bài báo gốc	7
1.4	Các công trình liên quan	8
1.4.1	Các bài toán định tuyến drone	8
1.4.2	Location Arc Routing Problem và bài toán nhiều depot	9
1.4.3	Các phương pháp tìm kiếm cục bộ cho bài toán định tuyến	9
1.5	Định hướng của đồ án	10
2	Phát biểu bài toán MM-MT-dLARP	11
2.1	Mô tả bài toán thực tế	11
2.2	Các thành phần của bài toán	12
2.2.1	Depot, xe tải và drone	12
2.2.2	Tập điểm triển khai	12
2.2.3	Tập đường cần phục vụ	12
2.2.4	Giới hạn thời lượng bay	13
2.3	Hàm mục tiêu min-max	13
2.4	Các ràng buộc chính	14
2.4.1	Ràng buộc phục vụ toàn bộ cạnh yêu cầu	14
2.4.2	Ràng buộc mỗi chuyến bay bắt đầu và kết thúc tại điểm triển khai	15
2.4.3	Ràng buộc giới hạn thời lượng bay	15
2.4.4	Ràng buộc số điểm triển khai được sử dụng	16
2.5	Rời rạc hóa bài toán	16
2.5.1	Từ MM-MT-dLARP sang MM-MT-LARP	16
2.5.2	Biểu diễn đường cong bằng chuỗi đa giác	17
2.5.3	Cạnh phục vụ và cạnh bay chết	17
2.6	Độ khó của bài toán	18
3	Biểu diễn nghiệm và các toán tử tìm kiếm cục bộ	20
3.1	Mục tiêu thiết kế	20
3.2	Dữ liệu sau rời rạc hóa	20

3.3	Biểu diễn nghiệm	21
3.4	Khởi tạo nghiệm	22
3.5	Các toán tử tìm kiếm cục bộ	22
3.5.1	Tối ưu trong một chuyến bay	22
3.5.2	Phá và sửa nghiệm	22
3.5.3	Dịch chuyển chuỗi $0-\ell$	23
3.5.4	Trao đổi chuỗi $\ell_1-\ell_2$	23
3.5.5	Tối ưu chuyến bay tại điểm nghẽn	23
3.6	Variable Neighborhood Descent	23
3.7	Variable Neighborhood Search	24
3.8	Large Neighborhood Search	24
3.9	Tương thích với pha chia nhỏ cạnh	24
4	Giải thuật Variable Neighborhood Search cho MM-MT-dLARP	26
4.1	Vai trò của VNS trong bài toán MM-MT-dLARP	26
4.2	Khởi tạo nghiệm	27
4.3	Các lân cận sử dụng trong VNS	27
4.3.1	Tối ưu bên trong một chuyến bay	27
4.3.2	Destroy-repair trên điểm triển khai nghẽn	27
4.3.3	Dịch chuyển chuỗi $0-\ell$	28
4.3.4	Trao đổi chuỗi $\ell_1-\ell_2$	28
4.4	VND như thủ tục tăng cường	28
4.5	Cơ chế shaking	29
4.6	Khung thuật toán VNS	29
4.7	Tham số chính	30
4.8	Nhận xét	30
5	Giải thuật Large Neighborhood Search cho MM-MT-dLARP	31
5.1	Vai trò của LNS trong bài toán MM-MT-dLARP	31
5.2	Khởi tạo nghiệm	32
5.3	Cơ chế destroy-repair trong LNS	32
5.3.1	Toán tử destroy	32
5.3.2	Toán tử repair	32
5.4	Tăng cường cục bộ sau repair	33
5.5	Quy tắc chấp nhận và cập nhật nghiệm	33
5.6	Pha chia nhỏ điểm phục vụ	34
5.7	Khung thuật toán LNS	34
5.8	Tham số chính	35
5.9	Nhận xét	35

6	Thực nghiệm và đánh giá kết quả	37
6.1	Mục tiêu thực nghiệm	37
6.2	Thiết lập thực nghiệm	37
6.3	Tập cấu hình thực nghiệm	38
6.4	Cấu hình tham số của các thuật toán	40
6.5	Cách tổng hợp kết quả	41
6.6	Kết quả tổng hợp	41
6.7	Phân tích kết quả	42
6.8	Nhận xét chung	43
7	Kết luận và hướng phát triển	44
7.1	Kết luận chung	44
7.2	Các kết quả đạt được	45
7.3	Hạn chế	45
7.4	Hướng phát triển	46
7.4.1	Cải tiến VNS	46
7.4.2	Phát triển LNS thích nghi	47
7.4.3	Chạy thực nghiệm thống kê rộng hơn	47
7.4.4	Kết hợp với cận dưới và mô hình exact	47
7.4.5	Cải tiến pha rời rạc hóa và chia nhỏ cạnh	48
7.4.6	Tối ưu hiệu năng cài đặt	48
7.5	Kết luận cuối cùng	48

Chương 1

Tổng quan bài toán và các công trình liên quan

1.1 Đặt vấn đề

Trong những năm gần đây, máy bay không người lái, hay drone, ngày càng được sử dụng rộng rãi trong các bài toán vận tải, giao hàng, giám sát và kiểm tra hạ tầng. So với các phương tiện mặt đất truyền thống, drone có lợi thế là có thể di chuyển trực tiếp trong không gian, không bị ràng buộc bởi mạng đường bộ, đồng thời có khả năng tiếp cận những khu vực khó đi lại hoặc nguy hiểm đối với con người. Vì vậy, drone đặc biệt phù hợp với các nhiệm vụ kiểm tra đường dây điện, đường ống, mạng lưới giao thông, kênh mương hoặc các tuyến hạ tầng dạng đường.

Trong các ứng dụng kiểm tra hạ tầng tuyến tính, đối tượng cần phục vụ thường không phải là các điểm rời rạc mà là các cạnh hoặc các đoạn tuyến liên tục. Do đó, lớp bài toán phù hợp để mô hình hóa các ứng dụng này là bài toán định tuyến trên cung, hay *Arc Routing Problem* (ARP), thay vì các bài toán định tuyến trên nút như Traveling Salesman Problem hoặc Vehicle Routing Problem. Trong bài toán định tuyến trên cạnh với drone, drone cần bay dọc theo các đoạn tuyến yêu cầu để thực hiện nhiệm vụ phục vụ, chẳng hạn như ghi hình, kiểm tra hoặc thu thập dữ liệu cảm biến.

Tuy nhiên, việc sử dụng drone cũng tạo ra nhiều thách thức mới. Thứ nhất, drone có giới hạn về thời lượng pin nên mỗi chuyến bay chỉ có thể kéo dài tối đa trong một khoảng thời gian hoặc quãng đường nhất định. Thứ hai, đối với các mạng hạ tầng lớn, một drone đơn lẻ không thể hoàn thành toàn bộ nhiệm vụ. Thứ ba, drone thường cần phương tiện mặt đất hỗ trợ để vận chuyển đến gần khu vực cần kiểm tra, thay pin, cất cánh và hạ cánh. Vì vậy, bài toán thực tế không chỉ là tìm đường bay cho drone, mà còn phải đồng thời quyết định vị trí triển khai phương tiện mặt đất, phân công các đoạn tuyến cần phục vụ cho từng drone, và xây dựng nhiều chuyến bay khả thi cho mỗi drone.

Từ những yêu cầu trên, bài toán *Min-Max Multi-Trip Drone Location Arc Routing Problem* (MM-MT-dLARP) được đề xuất nhằm mô hình hóa tình huống trong đó có một depot trung tâm, một tập các xe tải, mỗi xe tải mang theo một drone, và một tập các

điểm tiềm năng để xe tải có thể đến triển khai drone. Mỗi drone có thể thực hiện nhiều chuyến bay từ điểm triển khai của nó, nhưng mỗi chuyến bay phải thỏa mãn giới hạn về thời lượng bay. Mục tiêu của bài toán là hoàn thành toàn bộ nhiệm vụ kiểm tra trong thời gian sớm nhất, tức là tối thiểu hóa thời gian hoàn thành lớn nhất trong số các xe tải và drone tương ứng [5].

1.2 Mô tả khái quát bài toán MM-MT-dLARP

Bài toán MM-MT-dLARP xét một tập các đoạn tuyến cần được phục vụ bởi drone. Các đoạn tuyến này thường là các đường cong hoặc các đoạn hạ tầng liên tục trong không gian. Có một depot ban đầu, nơi xuất phát của P xe tải. Mỗi xe tải mang theo một drone và có thể di chuyển từ depot đến một điểm triển khai thuộc tập $\mathcal{D}$. Mỗi điểm triển khai có thể được hiểu là một vị trí vận hành, nơi xe tải dừng lại để drone cất cánh, hạ cánh và thay pin nếu cần.

Mỗi drone có thể thực hiện nhiều chuyến bay từ điểm triển khai đã chọn. Một chuyến bay của drone bắt đầu tại điểm triển khai, bay đến một hoặc nhiều đoạn tuyến cần phục vụ, thực hiện nhiệm vụ trên các đoạn tuyến đó, sau đó quay trở lại đúng điểm triển khai ban đầu. Do giới hạn pin, chi phí hoặc thời gian của mỗi chuyến bay không được vượt quá một hằng số L . Ngoài ra, hai xe tải khác nhau không được sử dụng cùng một điểm triển khai.

Mục tiêu của bài toán là lựa chọn P điểm triển khai trong tập $\mathcal{D}$, phân công các đoạn tuyến cần phục vụ cho các drone, và xây dựng tập các chuyến bay cho từng drone sao cho toàn bộ các đoạn tuyến yêu cầu được phục vụ. Hàm mục tiêu có dạng min-max: với mỗi xe tải, chi phí được tính bằng chi phí di chuyển từ depot đến điểm triển khai và quay lại, cộng với tổng chi phí các chuyến bay của drone tương ứng; bài toán cần tối thiểu hóa giá trị lớn nhất trong các chi phí này. Cách đặt mục tiêu như vậy phản ánh yêu cầu cân bằng tải giữa các xe tải và drone, vì thời gian hoàn thành toàn bộ nhiệm vụ phụ thuộc vào phương tiện hoàn thành muộn nhất.

Một điểm quan trọng của MM-MT-dLARP là bản chất liên tục của bài toán. Drone không bị giới hạn bởi mạng đường bộ và có thể đi trực tiếp giữa hai điểm bất kỳ trong không gian. Hơn nữa, drone có thể đi vào hoặc rời khỏi một đoạn tuyến tại các điểm nằm bên trong đoạn tuyến, không nhất thiết chỉ tại hai đầu mút. Đặc điểm này giúp tìm được nghiệm tốt hơn so với phương tiện mặt đất, nhưng cũng làm bài toán khó hơn do không gian lựa chọn là liên tục.

Để xử lý bài toán về mặt tính toán, bài báo gốc rời rạc hóa các đoạn tuyến liên tục bằng cách xấp xỉ mỗi đường bằng một chuỗi đa giác gồm hữu hạn các điểm trung gian. Khi đó, drone chỉ được phép đi vào hoặc rời khỏi một đoạn tuyến tại các điểm đã được chọn trước. Bài toán liên tục MM-MT-dLARP được chuyển thành phiên bản rời rạc gọi là *Min-Max Multi-Trip Location Arc Routing Problem* (MM-MT-LARP). Trong phiên bản rời rạc này, các đoạn nhỏ của chuỗi đa giác trở thành các cạnh yêu cầu trong đồ thị, còn các cạnh không yêu cầu biểu diễn việc drone bay trực tiếp giữa hai điểm mà không thực

hiện phục vụ.

1.3 Tổng quan về bài báo gốc

Bài báo *The Min-Max Multi-Trip Drone Location Arc Routing Problem* của Corberán, Plana và Sanchis là công trình nền tảng trực tiếp cho đề tài này [5]. Bài báo đưa ra một lớp bài toán mới kết hợp đồng thời ba yếu tố: định tuyến trên cung, lựa chọn vị trí triển khai, và phối hợp xe tải với drone có khả năng bay nhiều chuyến. Đây là sự kết hợp giữa bài toán Location Arc Routing Problem, bài toán nhiều depot hoặc nhiều điểm vận hành, và bài toán định tuyến drone có giới hạn thời lượng bay.

Đóng góp đầu tiên của bài báo là phát biểu chính thức bài toán MM-MT-dLARP và phiên bản rời rạc MM-MT-LARP. Trong mô hình rời rạc, bài toán được định nghĩa trên đồ thị vô hướng $G = (V, E)$, trong đó tập cạnh E gồm các cạnh yêu cầu E_R và các cạnh không yêu cầu E_{NR} . Các cạnh yêu cầu tương ứng với các đoạn cần drone bay qua để phục vụ, còn các cạnh không yêu cầu biểu diễn các đoạn bay chết, tức là bay từ điểm này sang điểm khác mà không phục vụ. Tập đỉnh bao gồm các điểm thuộc các đoạn tuyến, các điểm triển khai drone, và depot.

Đóng góp thứ hai là xây dựng mô hình quy hoạch nguyên cho MM-MT-LARP. Mô hình sử dụng các biến nhị phân để biểu diễn việc một chuyến bay có đi qua hoặc phục vụ một cạnh hay không, cùng với biến nhị phân biểu diễn việc một điểm triển khai có được mở hay không. Hàm mục tiêu là tối thiểu hóa biến t , đại diện cho chi phí lớn nhất trong các tuyến xe tải-drone. Các ràng buộc chính đảm bảo mỗi cạnh yêu cầu được phục vụ, mỗi chuyến bay là một chu trình liên thông bắt đầu và kết thúc tại điểm triển khai tương ứng, mỗi chuyến bay không vượt quá giới hạn L , và số điểm triển khai được sử dụng không vượt quá số xe tải P .

Đóng góp thứ ba là nghiên cứu đa diện và xây dựng các bất đẳng thức hợp lệ để tăng cường mô hình. Bài báo trình bày nhiều họ bất đẳng thức như ràng buộc liên thông, bất đẳng thức parity, bất đẳng thức (p)-connectivity và bất đẳng thức max-length. Một số họ bất đẳng thức được chứng minh là sinh facet cho một đa diện nối lỏng của bài toán. Trên cơ sở đó, các tác giả phát triển thuật toán branch-and-cut cho phiên bản rời rạc của bài toán.

Đóng góp thứ tư, và cũng là phần liên quan trực tiếp đến đề tài này, là thuật toán matheuristic cho MM-MT-dLARP. Do bài toán có độ khó cao, thuật toán exact chỉ áp dụng hiệu quả cho các thể hiện nhỏ. Vì vậy, bài báo đề xuất một thuật toán heuristic kết hợp giữa pha xây dựng nghiệm ban đầu, các thủ tục tìm kiếm cục bộ, Variable Neighborhood Descent (VND), và một pha chọn điểm trung gian trên các đoạn tuyến.

Cụ thể, trong pha xây dựng nghiệm, thuật toán chọn ngẫu nhiên P điểm triển khai, sau đó phân công các cạnh yêu cầu cho các điểm triển khai dựa trên xác suất tỉ lệ nghịch với khoảng cách. Với mỗi điểm triển khai, một giant tour được xây dựng theo chiến lược Nearest Neighbor, sau đó được tách thành nhiều chuyến bay khả thi sao cho mỗi chuyến không vượt quá giới hạn L . Sau khi có nghiệm ban đầu, thuật toán áp dụng bốn thủ tục

tìm kiếm cục bộ: destroy-and-repair move, intraroute move, 0-to- ℓ exchange, và ℓ_1 -to- ℓ_2 exchange. Các thủ tục này được tích hợp trong khung Variable Neighborhood Descent để cải thiện nghiệm.

Pha cuối cùng của matheuristic là pha *splitting*, trong đó thuật toán thêm các điểm trung gian vào các đoạn tuyến nhằm khai thác khả năng drone có thể đi vào hoặc rời khỏi một đường tại các điểm nằm bên trong đường. Ban đầu, mỗi đường có thể được biểu diễn bằng một đoạn duy nhất. Sau đó, thuật toán thử thêm trung điểm vào các cạnh, chạy lại VND để kiểm tra xem các điểm trung gian mới có giúp cải thiện nghiệm hay không, rồi giữ lại các điểm hữu ích và loại bỏ các điểm không được sử dụng.

Các thực nghiệm trong bài báo được tiến hành trên tập thể hiện sinh ngẫu nhiên, với tối đa 6 điểm triển khai và 88 đường gốc. Kết quả cho thấy branch-and-cut chỉ phù hợp với các thể hiện nhỏ, trong khi matheuristic có khả năng tìm nghiệm chất lượng tốt trong thời gian hợp lý. Điều này cho thấy đối với MM-MT-dLARP, các phương pháp tìm kiếm cục bộ và metaheuristic đóng vai trò quan trọng khi kích thước bài toán tăng lên.

1.4 Các công trình liên quan

1.4.1 Các bài toán định tuyến drone

Các nghiên cứu về định tuyến drone có thể chia thành hai nhóm chính. Nhóm thứ nhất là các bài toán giao hàng, trong đó drone phục vụ các khách hàng được mô hình hóa dưới dạng các nút. Nhóm này thường liên quan đến các biến thể của Vehicle Routing Problem hoặc Traveling Salesman Problem có drone hỗ trợ. Các tổng quan về drone-aided routing và phối hợp drone với xe tải được trình bày trong [8, 4].

Nhóm thứ hai là các bài toán kiểm tra hoặc giám sát mạng lưới, trong đó đối tượng phục vụ là các cạnh hoặc cung. Đây là nhóm có liên hệ trực tiếp hơn với MM-MT-dLARP. Campbell và cộng sự đề xuất bài toán Drone Arc Routing Problem, trong đó drone cần phục vụ một tập các đường liên tục và có thể đi vào hoặc rời khỏi đường tại các điểm trung gian [1]. Công trình này đặt nền móng cho việc mô hình hóa bài toán kiểm tra tuyến bằng drone dưới dạng bài toán định tuyến trên cung.

Các nghiên cứu tiếp theo mở rộng hướng này sang bài toán nhiều drone có giới hạn chiều dài chuyến bay. Campbell và cộng sự nghiên cứu bài toán *Length-Constrained (K)-Drones Rural Postman Problem*, trong đó một đội drone cần cùng nhau phục vụ các cạnh yêu cầu dưới ràng buộc độ dài mỗi chuyến bay [2]. Sau đó, các tác giả tiếp tục phân tích đa diện và đề xuất thuật toán mới cho bài toán này [3]. Các công trình này có liên hệ chặt chẽ với MM-MT-dLARP vì cùng xử lý đối tượng phục vụ là các cạnh và cùng xét ràng buộc thời lượng bay của drone.

Tuy nhiên, điểm khác biệt của MM-MT-dLARP là bài toán không chỉ thiết kế đường bay cho drone mà còn phải chọn các điểm triển khai cho xe tải. Do đó, bài toán kết hợp thêm quyết định location vào bài toán arc routing, đồng thời sử dụng mục tiêu min-max để cân bằng thời gian hoàn thành giữa các xe tải và drone.

1.4.2 Location Arc Routing Problem và bài toán nhiều depot

Location Arc Routing Problem là lớp bài toán kết hợp giữa quyết định mở cơ sở và quyết định định tuyến trên cung. Trong lớp bài toán này, ta không chỉ cần xác định các tuyến phục vụ các cạnh yêu cầu, mà còn phải quyết định các điểm xuất phát hoặc depot được sử dụng. MM-MT-dLARP có thể được xem là một biến thể đặc biệt của Location Arc Routing Problem, trong đó các cơ sở được mở là các điểm triển khai drone, còn các tuyến tương ứng là các chuyến bay của drone từ các điểm đó.

Bên cạnh yếu tố location, MM-MT-dLARP cũng có liên hệ với các bài toán nhiều depot. Trong các bài toán multi-depot arc routing, các phương tiện xuất phát từ nhiều depot khác nhau và cần phối hợp để phục vụ tập cạnh yêu cầu. Tuy nhiên, trong MM-MT-dLARP, các depot không hoàn toàn cố định từ đầu mà được chọn từ tập điểm triển khai tiềm năng. Ngoài ra, mỗi điểm triển khai gắn với một xe tải và một drone, trong đó drone có thể thực hiện nhiều chuyến bay. Điều này làm tăng độ phức tạp so với các bài toán nhiều depot cổ điển.

1.4.3 Các phương pháp tìm kiếm cục bộ cho bài toán định tuyến

Do các bài toán định tuyến thường có độ phức tạp tổ hợp cao, các phương pháp tìm kiếm cục bộ và metaheuristic được sử dụng rộng rãi để tìm nghiệm tốt trong thời gian hợp lý. Tìm kiếm cục bộ bắt đầu từ một nghiệm khả thi và lặp lại việc di chuyển sang nghiệm lân cận tốt hơn theo một tập phép biến đổi được định nghĩa trước. Chất lượng của thuật toán phụ thuộc nhiều vào cách mã hóa nghiệm, cách định nghĩa lân cận, chiến lược chọn nước đi và cơ chế thoát khỏi cực trị địa phương.

Variable Neighborhood Search (VNS) là một metaheuristic dựa trên ý tưởng thay đổi có hệ thống các cấu trúc lân cận trong quá trình tìm kiếm [9]. Thay vì chỉ sử dụng một loại lân cận cố định, VNS khai thác nhiều lân cận khác nhau để tăng khả năng thoát khỏi cực trị địa phương. Một biến thể quan trọng của VNS là Variable Neighborhood Descent (VND), trong đó thuật toán lần lượt áp dụng các thủ tục tìm kiếm cục bộ trên các lân cận khác nhau cho đến khi không còn cải thiện. Các nguyên lý và ứng dụng của VNS được trình bày trong [6, 7].

Large Neighborhood Search (LNS) là một hướng tiếp cận khác, trong đó thuật toán phá hủy một phần nghiệm hiện tại rồi xây dựng lại phần bị phá hủy để tạo nghiệm mới [12]. Cơ chế destroy-and-repair giúp LNS khám phá các lân cận lớn hơn nhiều so với các phép hoán đổi nhỏ truyền thống. Adaptive Large Neighborhood Search (ALNS) mở rộng LNS bằng cách sử dụng nhiều toán tử destroy và repair khác nhau, đồng thời điều chỉnh xác suất chọn toán tử dựa trên hiệu quả của chúng trong quá trình tìm kiếm [11, 10].

Trong bài báo gốc về MM-MT-dLARP, các tác giả đã sử dụng một khung VND gồm bốn thủ tục cải thiện nghiệm. Trong đó, destroy-and-repair move có bản chất gần với LNS vì nó xóa một số cạnh yêu cầu khỏi nghiệm hiện tại rồi thử chèn lại vào các vị trí tốt hơn. Tuy nhiên, thuật toán trong bài báo chưa phải là một VNS hoặc LNS đầy đủ theo nghĩa cổ điển. Đây chính là cơ sở để đề tài này tập trung nghiên cứu và phát triển các giải thuật

tìm kiếm cục bộ, VNS và LNS cho bài toán MM-MT-dLARP.

1.5 Định hướng của đề án

Dựa trên bài báo gốc, đề án này tập trung vào khía cạnh thuật toán tìm kiếm cục bộ cho bài toán MM-MT-dLARP. Thay vì đi sâu vào mô hình quy hoạch nguyên và branch-and-cut, đề án hướng đến việc phân tích cách biểu diễn nghiệm, các cấu trúc lân cận, và khả năng xây dựng các khung metaheuristic như VNS và LNS.

Cụ thể, đề án xem xét nghiệm của bài toán dưới dạng tập các điểm triển khai được chọn và tập các chuyến bay của drone tại mỗi điểm triển khai. Một nghiệm tốt cần thỏa mãn đồng thời ba yếu tố: mỗi cạnh yêu cầu được phục vụ, mỗi chuyến bay không vượt quá giới hạn L , và giá trị makespan được giảm thiểu. Từ biểu diễn này, có thể xây dựng nhiều nhóm toán tử lân cận như di chuyển một cạnh yêu cầu giữa hai chuyến bay, đổi chỗ hai chuỗi cạnh giữa hai chuyến bay, tối ưu lại thứ tự phục vụ trong một chuyến bay, hoặc phá hủy và tái xây dựng một phần nghiệm tại điểm triển khai đang là nút thắt.

Đối với hướng VNS, đề án có thể định nghĩa nhiều cấu trúc lân cận $N_1, N_2, \dots, N_k$, ví dụ như relocate, swap, sequence exchange và destroy-and-repair. Quá trình tìm kiếm sẽ thay đổi lân cận một cách có hệ thống, kết hợp bước shaking để tạo nhiễu và bước local search để cải thiện nghiệm. Đối với hướng LNS, đề án có thể tập trung vào các chiến lược destroy-and-repair chuyên biệt cho MM-MT-dLARP, chẳng hạn ưu tiên phá hủy tại điểm triển khai có makespan lớn nhất, xóa các cạnh có chi phí chèn cao, hoặc tái chèn các cạnh theo tiêu chí giảm bottleneck.

Như vậy, chương này đã trình bày tổng quan về bối cảnh ứng dụng, mô tả khái quát bài toán MM-MT-dLARP, tóm tắt nội dung chính của bài báo nền tảng và các hướng nghiên cứu liên quan. Các chương tiếp theo sẽ đi sâu hơn vào mô hình bài toán, cách mã hóa nghiệm, các toán tử lân cận và thiết kế thuật toán tìm kiếm cục bộ cho bài toán này.

Chương 2

Phát biểu bài toán MM-MT-dLARP

2.1 Mô tả bài toán thực tế

Bài toán *Min-Max Multi-Trip Drone Location Arc Routing Problem*, viết tắt là MM-MT-dLARP, được đề xuất trong nghiên cứu của Corberán, Plana và Sanchis [5]. Đây là một biến thể của bài toán định tuyến trên cung, trong đó nhiệm vụ chính không phải là ghé thăm các điểm rời rạc, mà là phục vụ một tập các đường hoặc đoạn tuyến cần được bay qua bởi drone. Các đường này có thể đại diện cho đường dây điện, đường ống, tuyến đường giao thông, kênh mương hoặc các hạ tầng tuyến tính cần được kiểm tra, giám sát hoặc ghi hình.

Trong thực tế, drone thường có lợi thế lớn so với phương tiện mặt đất vì có thể bay trực tiếp giữa hai vị trí bất kỳ, không bị giới hạn bởi mạng đường bộ. Khi kiểm tra một tuyến hạ tầng, drone chỉ cần bay dọc theo tuyến đó để thu thập dữ liệu, chẳng hạn hình ảnh hoặc video. Tuy nhiên, drone có giới hạn về pin, do đó không thể tự thực hiện toàn bộ nhiệm vụ nếu mạng cần kiểm tra có quy mô lớn. Vì vậy, bài toán xem xét thêm sự hỗ trợ của xe tải. Mỗi xe tải mang theo một drone, xuất phát từ depot, di chuyển đến một điểm triển khai, sau đó drone được phóng từ điểm này để thực hiện một hoặc nhiều chuyến bay phục vụ các đường yêu cầu.

Điểm quan trọng của bài toán là phải đồng thời quyết định:

- chọn các điểm triển khai nào cho các xe tải;
- phân công các đường hoặc đoạn đường cần phục vụ cho từng drone;
- chia nhiệm vụ của mỗi drone thành nhiều chuyến bay khả thi;
- xác định thứ tự và hướng phục vụ các cạnh trong từng chuyến bay;
- tối thiểu hóa thời gian hoàn thành lớn nhất trong toàn hệ thống.

Do đó, MM-MT-dLARP kết hợp ba thành phần khó của tối ưu tổ hợp: bài toán chọn vị trí triển khai, bài toán định tuyến trên cạnh và bài toán cân bằng tải theo mục tiêu

min-max. Trong phạm vi đồ án này, bài toán được sử dụng làm nền tảng để xây dựng và đánh giá các giải thuật tìm kiếm cục bộ, đặc biệt là VNS và LNS.

2.2 Các thành phần của bài toán

2.2.1 Depot, xe tải và drone

Bài toán có một depot, ký hiệu là 0, là nơi các xe tải xuất phát và quay về sau khi hoàn thành nhiệm vụ. Có P xe tải, mỗi xe tải mang theo một drone. Mỗi xe tải được gán cho nhiều nhất một điểm triển khai, và tại điểm đó drone có thể thực hiện nhiều chuyến bay liên tiếp.

Khi một xe tải được gán đến điểm triển khai d , chi phí di chuyển của xe tải thường được tính là chi phí đi từ depot đến d và quay lại depot. Nếu ký hiệu c_{0d} là chi phí di chuyển một chiều từ depot đến điểm triển khai d , thì chi phí xe tải tương ứng là:

$$2c_{0d}. \quad (2.1)$$

Drone được phóng từ điểm triển khai, thực hiện một chuyến bay để phục vụ một số cạnh yêu cầu, sau đó quay lại đúng điểm triển khai ban đầu. Sau mỗi chuyến bay, drone có thể được thay pin hoặc sạc lại, vì vậy một drone có thể thực hiện nhiều chuyến bay. Đây là lý do bài toán có cụm từ *multi-trip*.

2.2.2 Tập điểm triển khai

Gọi $\mathcal{D}$ là tập các điểm triển khai tiềm năng, với $D = |\mathcal{D}|$. Các điểm này là những vị trí mà xe tải có thể dừng lại để phóng, thu hồi, thay pin hoặc xử lý drone. Số điểm triển khai tiềm năng thường lớn hơn hoặc bằng số xe tải:

$$D \geq P. \quad (2.2)$$

Mỗi điểm triển khai có thể được hiểu như một cơ sở tạm thời được mở nếu có xe tải được điều đến đó. Hai xe tải không được sử dụng cùng một điểm triển khai. Vì vậy, bài toán không chỉ là bài toán định tuyến, mà còn chứa quyết định chọn vị trí. Nếu ký hiệu z_d là biến nhị phân cho biết điểm triển khai d có được sử dụng hay không, ta có:

$$z_d = \begin{cases} 1, & \text{nếu điểm triển khai } d \text{ được sử dụng,} \\ 0, & \text{ngược lại.} \end{cases} \quad (2.3)$$

2.2.3 Tập đường cần phục vụ

Đối tượng phục vụ của bài toán là một tập các đường, thường là các đường cong trong mặt phẳng. Mỗi đường cần được drone bay qua toàn bộ để thực hiện nhiệm vụ kiểm tra

hoặc giám sát. Khác với bài toán định tuyến nút, ở đây yêu cầu không nằm tại các đỉnh mà nằm trên các cạnh hoặc đoạn tuyến. Vì vậy, bài toán thuộc nhóm bài toán định tuyến trên cung.

Trong mô hình liên tục MM-MT-dLARP, drone có thể đi vào một đường tại bất kỳ điểm nào, phục vụ một phần của đường, sau đó rời khỏi đường tại một điểm khác. Điều này phản ánh đúng khả năng bay của drone, vì drone có thể di chuyển ngoài mạng và không bắt buộc phải đi theo các cạnh có sẵn như phương tiện mặt đất. Tuy nhiên, chính đặc điểm này cũng làm cho bài toán trở thành một bài toán tối ưu liên tục khó giải.

Sau khi rời rạc hóa, tập đường cần phục vụ được biểu diễn thành tập cạnh yêu cầu, ký hiệu là E_R . Mỗi cạnh yêu cầu $e \in E_R$ có chi phí phục vụ $c_e^s > 0$. Một cạnh yêu cầu chỉ được xem là đã hoàn thành khi drone bay dọc theo cạnh đó ở trạng thái phục vụ, không phải chỉ bay qua như một đoạn di chuyển chết.

2.2.4 Giới hạn thời lượng bay

Mỗi chuyến bay của drone bị giới hạn bởi thời lượng hoặc quãng đường bay tối đa, ký hiệu là L . Đây là tham số thể hiện dung lượng pin hoặc giới hạn vận hành an toàn của drone. Nếu một chuyến bay vượt quá L , chuyến bay đó không khả thi.

Giới hạn L được áp dụng cho từng chuyến bay riêng lẻ, không phải cho tổng thời gian hoạt động của drone tại một điểm triển khai. Nói cách khác, một drone có thể thực hiện nhiều chuyến bay, nhưng mỗi chuyến bay phải thỏa mãn:

$$\text{cost}(\text{flight}) \leq L. \quad (2.4)$$

Tổng chi phí của tất cả các chuyến bay từ cùng một điểm triển khai vẫn được dùng để tính thời gian hoàn thành của xe tải-drone tương ứng trong hàm mục tiêu min-max.

2.3 Hàm mục tiêu min-max

Mục tiêu của MM-MT-dLARP là tối thiểu hóa thời gian hoàn thành lớn nhất trong các xe tải-drone. Với mỗi điểm triển khai d , nếu điểm này được sử dụng, tổng thời gian tương ứng bao gồm:

- thời gian xe tải đi từ depot đến điểm triển khai và quay về;
- tổng thời gian của tất cả các chuyến bay drone xuất phát từ điểm triển khai đó.

Gọi $\mathcal{K} = \{1, 2, \dots, K\}$ là tập chỉ số các chuyến bay tối đa có thể xét cho mỗi điểm triển khai. Với mỗi $d \in \mathcal{D}$ và $k \in \mathcal{K}$, gọi C_{dk} là chi phí của chuyến bay thứ k xuất phát từ điểm triển khai d . Khi đó tổng chi phí gắn với điểm triển khai d là:

$$T_d = 2c_{0d}z_d + \sum_{k \in \mathcal{K}} C_{dk}. \quad (2.5)$$

Hàm mục tiêu min–max được viết dưới dạng:

$$\min \max_{d \in \mathcal{D}} T_d. \quad (2.6)$$

Để đưa về dạng tuyến tính, ta sử dụng biến phụ t biểu diễn makespan, tức thời gian hoàn thành lớn nhất. Khi đó hàm mục tiêu trở thành:

$$\min t, \quad (2.7)$$

với các ràng buộc:

$$2c_{0d}z_d + \sum_{k \in \mathcal{K}} C_{dk} \leq t, \quad \forall d \in \mathcal{D}. \quad (2.8)$$

Ý nghĩa của hàm mục tiêu này là cân bằng tải giữa các xe tải–drone. Một nghiệm tốt không chỉ có tổng chi phí nhỏ, mà còn tránh trường hợp một drone phải thực hiện quá nhiều nhiệm vụ trong khi các drone khác gần như không hoạt động.

Trong đồ án này, giá trị t cũng là tiêu chí đánh giá chính của các giải thuật tìm kiếm cục bộ. Khi một phép biến đổi nghiệm được thực hiện, nghiệm mới được xem là cải thiện nếu làm giảm makespan hoặc cải thiện cấu trúc nghiệm mà không vi phạm các ràng buộc khả thi.

2.4 Các ràng buộc chính

2.4.1 Ràng buộc phục vụ toàn bộ cạnh yêu cầu

Mỗi cạnh yêu cầu phải được phục vụ ít nhất một lần bởi một chuyến bay drone nào đó. Trong mô hình rời rạc, gọi x_e^{dk} là biến nhị phân nhận giá trị 1 nếu cạnh e được đi qua trong chuyến bay k xuất phát từ điểm triển khai d . Với cạnh yêu cầu $e \in E_R$, biến $x_e^{dk} = 1$ thể hiện rằng cạnh này được drone phục vụ trong chuyến bay tương ứng.

Ràng buộc phục vụ toàn bộ cạnh yêu cầu được viết là:

$$\sum_{d \in \mathcal{D}} \sum_{k \in \mathcal{K}} x_e^{dk} \geq 1, \quad \forall e \in E_R. \quad (2.9)$$

Trong một số cài đặt thuật toán, ràng buộc này có thể được duy trì dưới dạng bằng:

$$\sum_{d \in \mathcal{D}} \sum_{k \in \mathcal{K}} x_e^{dk} = 1, \quad \forall e \in E_R, \quad (2.10)$$

vì việc phục vụ lặp lại một cạnh yêu cầu thường làm tăng chi phí và không mang lại lợi ích. Tuy nhiên, trong mô hình tổng quát của bài báo, dạng bất đẳng thức được dùng để thuận lợi cho nghiên cứu đa diện và thuật toán branch-and-cut [5].

2.4.2 Ràng buộc mỗi chuyến bay bắt đầu và kết thúc tại điểm triển khai

Mỗi chuyến bay drone phải là một chu trình đóng bắt đầu và kết thúc tại cùng một điểm triển khai. Điều này bảo đảm drone được thu hồi tại đúng vị trí xe tải đang chờ.

Trong biểu diễn đồ thị, một chuyến bay hợp lệ phải thỏa mãn hai điều kiện chính. Thứ nhất, bậc của mỗi đỉnh trong đồ thị con ứng với chuyến bay phải là số chẵn, vì drone đi vào và đi ra khỏi mỗi đỉnh cùng số lần. Điều kiện này có thể biểu diễn dưới dạng:

$$(x^{dk} + y^{dk})(\delta(v)) \equiv 0 \pmod{2}, \quad \forall v \in V, \forall d \in \mathcal{D}, \forall k \in \mathcal{K}, \quad (2.11)$$

trong đó y_e^{dk} là biến biểu diễn lần đi qua thứ hai trên cạnh bay chết, và $\delta(v)$ là tập các cạnh kề với đỉnh v .

Thứ hai, đồ thị con của chuyến bay phải liên thông với điểm triển khai d . Nếu một nhóm cạnh được chọn nhưng không nối với điểm triển khai, ta sẽ thu được một chu trình con không thể thực hiện trong chuyến bay thực tế. Ràng buộc loại bỏ chu trình con có thể viết dưới dạng:

$$(x^{dk} + y^{dk})(\delta(S)) \geq 2x_f^{dk}, \quad \forall S \subseteq V \setminus \{d\}, \quad \forall f \in E(S), \forall d \in \mathcal{D}, \forall k \in \mathcal{K}. \quad (2.12)$$

Về mặt thuật toán tìm kiếm cục bộ, thay vì kiểm tra trực tiếp các bất đẳng thức trên, một chuyến bay thường được biểu diễn bằng một chuỗi có thứ tự các cạnh yêu cầu. Khi tính chi phí chuyến bay, ta thêm chi phí bay chết từ điểm triển khai đến cạnh đầu tiên, giữa các cạnh liên tiếp, và từ cạnh cuối cùng quay về điểm triển khai. Cách biểu diễn này tự nhiên bảo đảm mỗi chuyến bay bắt đầu và kết thúc tại đúng điểm triển khai.

2.4.3 Ràng buộc giới hạn thời lượng bay

Mỗi chuyến bay phải có chi phí không vượt quá giới hạn L . Trong mô hình rời rạc, ràng buộc giới hạn thời lượng bay của chuyến bay k xuất phát từ điểm triển khai d được viết là:

$$\sum_{e \in E_R} c_e^s x_e^{dk} + \sum_{e \in E_{NR}} c_e (x_e^{dk} + y_e^{dk}) \leq Lz^d, \quad \forall dk \in \mathcal{D} \times \mathcal{K}. \quad (2.13)$$

Trong đó:

- E_R là tập cạnh yêu cầu;
- E_{NR} là tập cạnh bay chết;
- c_e^s là chi phí phục vụ cạnh yêu cầu e ;
- c_e là chi phí bay chết trên cạnh e ;
- x_e^{dk} và y_e^{dk} là các biến quyết định liên quan đến việc chuyến bay k tại điểm triển khai d đi qua cạnh e ;

- z^d là biến cho biết điểm triển khai d có được sử dụng hay không.

Nếu $z^d = 0$, nghĩa là điểm triển khai d không được sử dụng, ràng buộc trên buộc mọi chuyến bay xuất phát từ d phải rỗng. Nếu $z^d = 1$, mỗi chuyến bay từ d được phép hoạt động nhưng phải có chi phí không vượt quá L .

2.4.4 Ràng buộc số điểm triển khai được sử dụng

Vì có P xe tải, số điểm triển khai được sử dụng không được vượt quá P . Ràng buộc này được biểu diễn bởi:

$$\sum_{d \in \mathcal{D}} z_d \leq P. \quad (2.14)$$

Trong trường hợp yêu cầu sử dụng đúng P xe tải, ràng buộc trên có thể được thay bằng:

$$\sum_{d \in \mathcal{D}} z_d = P. \quad (2.15)$$

Tuy nhiên, dạng $\leq P$ linh hoạt hơn và phù hợp với mô hình trong bài báo. Nếu một điểm triển khai không được chọn, tất cả các chuyến bay gắn với điểm đó đều rỗng. Nếu một điểm triển khai được chọn, nó tương ứng với một xe tải và một drone thực hiện một hoặc nhiều chuyến bay.

2.5 Rời rạc hóa bài toán

2.5.1 Từ MM-MT-dLARP sang MM-MT-LARP

MM-MT-dLARP là bài toán liên tục vì drone có thể đi vào hoặc rời khỏi một đường tại bất kỳ điểm nào trên đường đó. Để xây dựng mô hình toán và thuật toán, bài báo rời rạc hóa bài toán bằng cách xấp xỉ mỗi đường cong bởi một chuỗi đa giác hữu hạn. Sau bước này, drone chỉ được phép đi vào hoặc rời khỏi đường tại các điểm đã được chọn trong chuỗi đa giác.

Bài toán rời rạc thu được được gọi là *Min-Max Multi-Trip Location Arc Routing Problem*, viết tắt là MM-MT-LARP. Bài toán này được định nghĩa trên một đồ thị vô hướng:

$$G = (V, E). \quad (2.16)$$

Tập đỉnh V bao gồm:

- các đỉnh nằm trên các đường cần phục vụ sau khi rời rạc hóa;
- các điểm triển khai trong $\mathcal{D}$;
- depot hoặc thông tin chi phí từ depot đến các điểm triển khai.

Tập cạnh E được chia thành hai nhóm:

$$E = E_R \cup E_{NR}, \quad (2.17)$$

trong đó E_R là tập cạnh yêu cầu cần được phục vụ, còn E_{NR} là tập cạnh không yêu cầu dừng cho di chuyển chết của drone.

Rời rạc hóa giúp chuyển bài toán từ miền liên tục sang bài toán tối ưu tổ hợp trên đồ thị. Tuy nhiên, nếu số điểm trung gian trên các đường quá lớn, kích thước đồ thị tăng nhanh, đặc biệt vì tập cạnh bay chết thường được xây dựng gần như đầy đủ giữa các cặp đỉnh. Do đó, việc lựa chọn số lượng và vị trí điểm trung gian là một yếu tố quan trọng trong thiết kế thuật toán.

2.5.2 Biểu diễn đường cong bằng chuỗi đa giác

Giả sử một đường cong cần phục vụ được ký hiệu là ℓ . Đường này được xấp xỉ bởi một dãy điểm:

$$p_0^\ell, p_1^\ell, \dots, p_{m_\ell}^\ell. \quad (2.18)$$

Các điểm liên tiếp tạo thành các đoạn thẳng:

$$(p_0^\ell, p_1^\ell), \quad (p_1^\ell, p_2^\ell), \quad \dots, \quad (p_{m_\ell-1}^\ell, p_{m_\ell}^\ell). \quad (2.19)$$

Mỗi đoạn thẳng này trở thành một cạnh yêu cầu trong tập E_R . Nếu chi phí phục vụ của đường gốc ℓ là c_ℓ^s , thì chi phí phục vụ của các đoạn con được phân bổ sao cho tổng chi phí của chúng bằng chi phí phục vụ đường gốc:

$$\sum_{e \in E_R(\ell)} c_e^s = c_\ell^s, \quad (2.20)$$

trong đó $E_R(\ell)$ là tập các cạnh yêu cầu sinh ra từ đường ℓ .

Cách biểu diễn này cho phép drone phục vụ từng phần của đường thay vì bắt buộc phải đi từ đầu đến cuối đường gốc. Nếu một điểm trung gian được chọn làm điểm vào hoặc điểm ra, drone có thể phục vụ một đoạn con, rời khỏi đường, bay chết đến một vị trí khác và tiếp tục phục vụ đoạn khác. Đây là đặc trưng quan trọng giúp drone tạo ra các nghiệm linh hoạt hơn so với phương tiện mặt đất.

2.5.3 Cạnh phục vụ và cạnh bay chết

Trong MM-MT-LARP, cần phân biệt rõ hai loại cạnh: cạnh phục vụ và cạnh bay chết.

Cạnh phục vụ là các cạnh thuộc E_R . Khi drone bay dọc theo cạnh này ở trạng thái phục vụ, nhiệm vụ kiểm tra hoặc giám sát trên cạnh đó được hoàn thành. Chi phí tương ứng là c_e^s .

Cạnh bay chết là các cạnh thuộc E_{NR} . Đây là các đoạn di chuyển của drone khi không thực hiện phục vụ, chẳng hạn:

- bay từ điểm triển khai đến cạnh yêu cầu đầu tiên;
- bay giữa hai cạnh yêu cầu không kề nhau;
- bay từ cạnh yêu cầu cuối cùng quay về điểm triển khai.

Vì drone có thể bay trực tiếp giữa hai điểm bất kỳ, tập cạnh bay chết thường chứa cạnh giữa mọi cặp đỉnh trong V . Chi phí bay chết thường được tính tỉ lệ với khoảng cách Euclid giữa hai điểm:

$$c_{ij} \propto \text{dist}(i, j). \quad (2.21)$$

Với mỗi cạnh yêu cầu $e \in E_R$, mô hình cũng xét một cạnh bay chết song song $e' \in E_{NR}$ nối cùng hai đầu mút. Điều này cho phép phân biệt hai tình huống:

- drone bay dọc cạnh e để phục vụ, chịu chi phí c_e^s ;
- drone bay qua cùng đoạn hình học nhưng không phục vụ, chịu chi phí bay chết $c_{e'}$.

Thông thường ta có:

$$c_e^s \geq c_{e'}, \quad (2.22)$$

vì bay phục vụ có thể chậm hơn hoặc tốn chi phí hơn so với bay chết.

2.6 Độ khó của bài toán

MM-MT-dLARP là một bài toán khó do kết hợp nhiều tầng quyết định phụ thuộc lẫn nhau. Thứ nhất, bài toán phải chọn tối đa P điểm triển khai trong tập $\mathcal{D}$. Đây là thành phần của bài toán vị trí. Thứ hai, các cạnh yêu cầu phải được phân công cho các drone và các chuyến bay cụ thể. Đây là thành phần phân cụm và phân tải. Thứ ba, với mỗi chuyến bay, cần tìm thứ tự phục vụ và hướng đi của các cạnh sao cho chi phí nhỏ và không vượt quá giới hạn L . Đây là thành phần định tuyến trên cung.

Ngoài ra, hàm mục tiêu min-max làm bài toán khó hơn so với mục tiêu tối thiểu hóa tổng chi phí. Một phép di chuyển có thể làm giảm tổng chi phí nhưng không cải thiện makespan nếu không tác động đến điểm triển khai đang có tải lớn nhất. Vì vậy, các giải thuật tìm kiếm cục bộ cho bài toán này cần tập trung nhiều vào điểm nghẽn, tức xe tải-drone có tổng thời gian lớn nhất.

Kích thước mô hình rời rạc cũng tăng rất nhanh. Nếu số đỉnh sau rời rạc hóa là $|V|$, tập cạnh bay chết có thể có kích thước bậc $O(|V|^2)$ do drone có thể bay trực tiếp giữa nhiều cặp điểm. Khi xét thêm D điểm triển khai và tối đa K chuyến bay cho mỗi điểm triển khai, số biến định tuyến tăng theo bậc:

$$O(DK(|E_R| + |E_{NR}|)). \quad (2.23)$$

Điều này giải thích vì sao phương pháp exact như branch-and-cut chỉ phù hợp với các thể hiện nhỏ. Trong nghiên cứu gốc, thuật toán branch-and-cut chỉ được áp dụng hiệu

quả cho các thể hiện nhỏ của MM-MT-LARP không xét điểm trung gian, trong khi thuật toán matheuristic được dùng để xử lý các thể hiện lớn hơn của MM-MT-dLARP [5].

Từ góc nhìn của đồ án, độ khó này là động lực để nghiên cứu các giải thuật tìm kiếm cục bộ như VNS và LNS. Thay vì giải chính xác toàn bộ mô hình, các phương pháp này cải thiện dần nghiệm thông qua các thao tác như chuyển cạnh giữa các chuyến bay, hoán đổi chuỗi cạnh, phá hủy và sửa chữa một phần nghiệm, hoặc tập trung tối ưu điểm triển khai đang gây ra makespan lớn nhất. Cách tiếp cận này phù hợp với bản chất của MM-MT-dLARP, nơi một nghiệm có thể được cải thiện đáng kể bằng các thay đổi cục bộ nhưng có tác động trực tiếp đến sự cân bằng tải giữa các drone.

Chương 3

Biểu diễn nghiệm và các toán tử tìm kiếm cục bộ

3.1 Mục tiêu thiết kế

Bài toán MM-MT-dLARP kết hợp hai quyết định: chọn các điểm triển khai drone cho xe tải và phân chia các cạnh yêu cầu cho các chuyến bay của drone. Theo mô hình gốc, các đường cần phục vụ có thể là đường cong và được rời rạc hóa thành các chuỗi đoạn thẳng; drone chỉ đi vào hoặc rời một đường tại các điểm rời rạc đã chọn [5]. Vì vậy, các giải thuật tìm kiếm cục bộ trong báo cáo này không thao tác trực tiếp trên hình học liên tục, mà thao tác trên một phiên bản rời rạc của bài toán.

Thiết kế nghiệm được chọn theo ba nguyên tắc. Thứ nhất, một cạnh yêu cầu phải được phục vụ đúng một lần. Thứ hai, mỗi chuyến bay phải bắt đầu và kết thúc tại cùng một điểm triển khai, đồng thời thỏa giới hạn bay của drone. Thứ ba, vì hàm mục tiêu là min-max, các toán tử ưu tiên cải thiện tuyến đang tạo ra giá trị lớn nhất của nghiệm.

3.2 Dữ liệu sau rời rạc hóa

Gọi $G = (V, E)$ là đồ thị sau rời rạc hóa. Tập cạnh yêu cầu cần được drone bay qua là $R \subseteq E$. Mỗi cạnh $e \in R$ có hai đầu mút u_e, v_e và chi phí phục vụ s_e . Tập điểm triển khai ứng viên là $D \subseteq V$, depot của xe tải là 0, số xe tải tối đa là m , và giới hạn mỗi chuyến bay của drone là L .

Khoảng cách di chuyển không phục vụ giữa hai đỉnh $i, j \in V$ được ký hiệu là c_{ij} . Một cạnh yêu cầu có thể được phục vụ theo một trong hai hướng. Do đó, một nhiệm vụ phục vụ được viết dưới dạng

$$\tau = (e, \sigma), \quad e \in R, \quad \sigma \in \{+, -\},$$

trong đó σ xác định drone đi từ u_e sang v_e hay ngược lại.

3.3 Biểu diễn nghiệm

Một nghiệm S được biểu diễn bởi

$$S = (D(S), \{\mathcal{F}_d\}_{d \in D(S)}),$$

trong đó $D(S) \subseteq D$ là tập điểm triển khai được chọn và $|D(S)| \leq m$. Với mỗi điểm triển khai d , $\mathcal{F}_d$ là danh sách các chuyến bay của drone xuất phát từ d .

Một chuyến bay $F \in \mathcal{F}_d$ là một dãy có thứ tự các nhiệm vụ

$$F = (d; \tau_1, \tau_2, \dots, \tau_q; d).$$

Nếu $a(\tau)$ và $b(\tau)$ lần lượt là đỉnh bắt đầu và đỉnh kết thúc của nhiệm vụ τ , chi phí của chuyến bay được tính bởi

$$C(F) = c_{d,a(\tau_1)} + \sum_{i=1}^q s_{e_i} + \sum_{i=1}^{q-1} c_{b(\tau_i),a(\tau_{i+1})} + c_{b(\tau_q),d}.$$

Chuyến bay hợp lệ nếu

$$C(F) \leq L.$$

Chi phí tại điểm triển khai d gồm chi phí xe tải đi từ depot đến d và quay về, cộng với tổng chi phí các chuyến bay xuất phát từ d :

$$C_d(S) = 2c_{0d} + \sum_{F \in \mathcal{F}_d} C(F).$$

Hàm mục tiêu của bài toán là giảm tải lớn nhất trong các điểm triển khai được chọn:

$$f(S) = \max_{d \in D(S)} C_d(S).$$

Điểm triển khai gây ra giá trị này được gọi là điểm nghẽn của nghiệm:

$$d^*(S) = \arg \max_{d \in D(S)} C_d(S).$$

Một nghiệm được xem là khả thi khi thỏa đồng thời ba điều kiện: không dùng quá m điểm triển khai, mọi chuyến bay đều có chi phí không vượt quá L , và mỗi cạnh trong R xuất hiện đúng một lần trong toàn bộ các chuyến bay. Cách biểu diễn này đủ gọn để các toán tử có thể di chuyển, đổi hướng hoặc trao đổi nhiệm vụ mà không cần xây dựng lại toàn bộ nghiệm từ đầu.

3.4 Khởi tạo nghiệm

Nghiệm ban đầu được xây dựng theo hướng tham lam có ngẫu nhiên. Trước hết, thuật toán chọn một số điểm triển khai trong D , ưu tiên các điểm gần depot để không làm tăng chi phí xe tải. Sau đó, mỗi cạnh yêu cầu được gán cho một điểm triển khai gần nó. Với mỗi điểm triển khai, các cạnh đã gán được sắp thành một “giant tour” bằng quy tắc láng giềng gần nhất, rồi được cắt thành nhiều chuyến bay sao cho mỗi chuyến không vượt quá giới hạn L .

Việc sinh nhiều nghiệm ban đầu khác nhau giúp tìm kiếm cục bộ tránh phụ thuộc quá mạnh vào một cấu hình xuất phát. Sau khi sinh, mỗi nghiệm được đánh giá lại bằng hàm $f(S)$ và chỉ các nghiệm khả thi mới được giữ trong tập ứng viên.

3.5 Các toán tử tìm kiếm cục bộ

Các toán tử dưới đây đều nhận một nghiệm khả thi S và chỉ chấp nhận nghiệm mới S' nếu S' vẫn khả thi và $f(S') < f(S)$. Với các toán tử liên tuyến, vùng tìm kiếm thường tập trung vào điểm nghẽn $d^*(S)$ nhằm trực tiếp làm giảm thành phần đang quyết định hàm mục tiêu min-max.

3.5.1 Tối ưu trong một chuyến bay

Toán tử này chỉ thay đổi thứ tự và hướng phục vụ của các nhiệm vụ trong cùng một chuyến bay. Một nhiệm vụ được lấy ra khỏi vị trí hiện tại, thử chèn lại vào các vị trí khác với cả hai hướng phục vụ, sau đó giữ thay đổi tốt nhất nếu chi phí chuyến bay giảm và vẫn thỏa $C(F) \leq L$.

Toán tử này không làm thay đổi điểm triển khai hay phân bổ cạnh giữa các chuyến bay. Vai trò chính của nó là giảm chi phí di chuyển rỗng giữa các cạnh liên tiếp trong cùng một chuyến bay. Đây là lân cận nhỏ, rẻ, nên được ưu tiên dùng sớm trong VND và sau các bước sửa nghiệm của LNS.

3.5.2 Phá và sửa nghiệm

Toán tử phá-sửa lấy ra một số nhiệm vụ từ các chuyến bay tại điểm nghẽn $d^*(S)$. Các nhiệm vụ bị lấy ra được chèn lại lần lượt vào vị trí khả thi tốt nhất trong toàn bộ nghiệm: có thể vào một chuyến bay đang tồn tại, vào một vị trí khác trong cùng điểm triển khai, sang điểm triển khai khác, hoặc tạo một chuyến bay mới tại một điểm triển khai đã chọn.

Cách sửa nghiệm dựa trên chèn tốt nhất theo hàm mục tiêu hiện tại. Do đó, toán tử này có khả năng chuyển tải từ điểm nghẽn sang các điểm triển khai ít tải hơn, phù hợp với bản chất cân bằng tải của mục tiêu min-max.

3.5.3 Dịch chuyển chuỗi 0– ℓ

Toán tử 0– ℓ chọn một chuỗi gồm ℓ nhiệm vụ liên tiếp từ một chuyến bay tại điểm ghé và chuyển chuỗi này sang một chuyến bay khác. Chuyến bay nhận có thể thuộc cùng điểm triển khai hoặc điểm triển khai khác, miễn là không trùng với chính chuyến bay cho. Nếu cần, chuỗi nhiệm vụ cũng có thể tạo thành một chuyến bay mới.

Khác với phá-sửa, toán tử 0– ℓ giữ nguyên thứ tự tương đối của chuỗi nhiệm vụ được chuyển. Điều này giúp bảo toàn một đoạn đường bay đã tương đối tốt, đồng thời vẫn cho phép giảm tải tại điểm ghé.

3.5.4 Trao đổi chuỗi ℓ_1 – ℓ_2

Toán tử ℓ_1 – ℓ_2 hoán đổi một chuỗi ℓ_1 nhiệm vụ từ một chuyến bay tại điểm ghé với một chuỗi ℓ_2 nhiệm vụ từ một chuyến bay khác. Sau khi hoán đổi, nghiệm được đánh giá lại và chỉ giữ nếu tất cả chuyến bay vẫn thỏa giới hạn L .

Toán tử này mạnh hơn dịch chuyển một chiều vì nó thay đổi đồng thời hai phần của nghiệm. Trong thực nghiệm, ℓ_1 và ℓ_2 thường được giới hạn bởi một giá trị nhỏ $\ell_{\max}$ để giữ thời gian duyệt lân cận ở mức chấp nhận được.

3.5.5 Tối ưu chuyến bay tại điểm ghé

Sau khi các lân cận thông thường không còn cải thiện, thuật toán có thể tối ưu lại từng chuyến bay tại điểm ghé bằng một thủ tục chính xác hoặc gần chính xác trên tập nhiệm vụ cố định của chuyến bay đó. Khi số nhiệm vụ nhỏ, có thể xem đây là một bài toán định thứ tự có hai hướng phục vụ cho mỗi cạnh; khi số nhiệm vụ lớn hơn, có thể dùng thủ tục tối ưu hóa cục bộ rẻ hơn.

Bước này không thay đổi phân bố cạnh giữa các điểm triển khai, nhưng giúp kiểm tra xem giá trị tại điểm ghé còn có thể giảm chỉ bằng cách sắp xếp lại các nhiệm vụ hay không. Vì chi phí của thủ tục này cao hơn các toán tử khác, nó chỉ được gọi sau khi VND đã đạt cực tiểu cục bộ.

3.6 Variable Neighborhood Descent

Variable Neighborhood Descent (VND) dùng một danh sách lân cận có thứ tự và luôn quay lại lân cận đầu tiên khi tìm được cải thiện. Trong báo cáo này, thứ tự lân cận được dùng là

$\mathcal{N}_1$: tối ưu trong chuyến bay, $\mathcal{N}_2$: phá-sửa, $\mathcal{N}_3$: dịch chuyển 0– ℓ , $\mathcal{N}_4$: trao đổi ℓ_1 – ℓ_2 .

Ý tưởng của VND là ưu tiên các thay đổi rẻ và cục bộ trước, sau đó mới chuyển sang các thay đổi lớn hơn. Khi một lân cận tạo ra nghiệm tốt hơn, nghiệm hiện tại được cập nhật và quá trình quay lại $\mathcal{N}_1$. Nếu không lân cận nào cải thiện được nghiệm, nghiệm hiện

tại được xem là cực tiểu cục bộ theo toàn bộ danh sách lân cận. Cách tổ chức này phù hợp với nguyên lý của tìm kiếm lân cận biến đổi [9, 6, 7].

3.7 Variable Neighborhood Search

VNS mở rộng VND bằng một bước nhiễu nghiệm, thường gọi là *shaking*. Thay vì dừng tại cực tiểu cục bộ, thuật toán tạo một nghiệm lân cận xa hơn bằng cách áp dụng k lần phá-sửa ngẫu nhiên. Sau đó, nghiệm bị nhiễu được cải thiện lại bằng VND nhẹ. Nếu nghiệm mới tốt hơn nghiệm hiện tại, thuật toán chấp nhận nghiệm đó và đặt lại $k = 1$; ngược lại, k tăng dần cho đến $k_{\max}$.

Trong ngữ cảnh MM-MT-dLARP, VNS có vai trò thoát khỏi các cấu hình mà tải tại điểm nghẽn không thể giảm bằng một dịch chuyển nhỏ. Bước shaking tạo cơ hội thay đổi phân bố cạnh giữa các điểm triển khai, còn VND nhẹ nhanh chóng sắp xếp lại các chuyến bay sau khi nghiệm bị xáo trộn. Cuối quá trình, nghiệm tốt nhất được cải thiện thêm bằng VND đầy đủ.

3.8 Large Neighborhood Search

Large Neighborhood Search (LNS) tạo lân cận lớn bằng cách tạm thời loại bỏ một tỷ lệ ρ nhiệm vụ khỏi nghiệm hiện tại, rồi xây dựng lại phần bị thiếu bằng chiến lược chen tốt nhất. Tư tưởng này bắt nguồn từ việc kết hợp phá nghiệm và sửa nghiệm trong các bài toán định tuyến [12, 11, 10].

Với nghiệm S , bước phá chọn ngẫu nhiên khoảng $\rho|R|$ nhiệm vụ đang được phục vụ và loại chúng khỏi các chuyến bay. Bước sửa xét từng nhiệm vụ bị loại bỏ, thử mọi điểm triển khai, mọi chuyến bay, mọi vị trí chen và cả hai hướng phục vụ; phương án khả thi có giá trị mục tiêu tốt nhất được chọn. Sau khi sửa xong, nghiệm được cải thiện nhanh bằng toán tử tối ưu trong chuyến bay.

LNS khác VNS ở mức độ thay đổi nghiệm. VNS thường dùng shaking như một nhiễu có kiểm soát quanh nghiệm hiện tại, còn LNS phá một phần lớn hơn của nghiệm để tái phân bố lại tải. Vì vậy, LNS phù hợp khi nghiệm hiện tại bị kẹt do nhiều cạnh đã được gán sai điểm triển khai, khiến các toán tử nhỏ khó sửa từng bước.

3.9 Tương thích với pha chia nhỏ cạnh

Do bài toán gốc chứa các đường có thể là đường cong, việc rời rạc hóa ban đầu có thể chưa đủ mịn. Sau khi có một nghiệm tốt trên đồ thị thô, các cạnh đang được sử dụng có thể được chia nhỏ bằng cách thêm điểm giữa trên chuỗi đoạn thẳng. Nghiệm cũ được ánh xạ sang đồ thị mới bằng cách thay một cạnh cha bởi các cạnh con tương ứng, sau đó tiếp tục áp dụng các toán tử tìm kiếm cục bộ.

Cơ chế này giúp thuật toán cân bằng giữa tốc độ và độ chính xác. Ở giai đoạn đầu, nghiệm được tìm trên đồ thị nhỏ để giảm chi phí tính toán. Ở các giai đoạn sau, chỉ những vùng có khả năng cải thiện mới được làm mịn thêm. Biểu diễn nghiệm theo danh sách nhiệm vụ giúp việc chuyển đổi giữa hai mức rời rạc hóa tương đối tự nhiên: mỗi nhiệm vụ vẫn là một cạnh yêu cầu có hướng, chỉ khác ở mức chi tiết của cạnh đó.

Chương 4

Giải thuật Variable Neighborhood Search cho MM-MT-dLARP

4.1 Vai trò của VNS trong bài toán MM-MT-dLARP

Variable Neighborhood Search (VNS) là một khung tìm kiếm cục bộ dùng nhiều cấu trúc lân cận khác nhau để thoát khỏi cực trị cục bộ [9, 6, 7]. Với bài toán MM-MT-dLARP, một nghiệm không chỉ quyết định các điểm triển khai drone được mở mà còn quyết định cách chia các cạnh yêu cầu thành nhiều chuyến bay xuất phát và quay về cùng một điểm triển khai [5]. Do đó, không gian nghiệm có hai mức quyết định: mức phân bổ công việc giữa các điểm triển khai và mức thứ tự phục vụ trong từng chuyến bay.

Trong chương này, VNS được sử dụng như một thủ tục cải thiện nghiệm trên một thể hiện đã rời rạc hoá. Cho một nghiệm S , ký hiệu $D(S)$ là tập điểm triển khai được chọn và $\mathcal{F}_d(S)$ là tập các chuyến bay gắn với điểm triển khai d . Chi phí của một chuyến bay F được ký hiệu bởi $C(F)$, bao gồm chi phí di chuyển từ điểm triển khai, phục vụ các cạnh trong chuyến bay và quay về điểm triển khai. Hàm mục tiêu min-max được viết dưới dạng

$$f(S) = \max_{d \in D(S)} \left(2c_{0d} + \sum_{F \in \mathcal{F}_d(S)} C(F) \right), \quad (4.1)$$

trong đó c_{0d} là khoảng cách từ depot đến điểm triển khai d .

Do mục tiêu là cực đại theo điểm triển khai, mỗi nghiệm thường bị chi phối bởi một điểm triển khai nghẽn cổ chai. Ta gọi

$$b(S) = \arg \max_{d \in D(S)} \left(2c_{0d} + \sum_{F \in \mathcal{F}_d(S)} C(F) \right) \quad (4.2)$$

là điểm triển khai nghẽn của nghiệm S . Các toán tử chính của VNS ưu tiên tác động lên $b(S)$ nhằm giảm trực tiếp giá trị mục tiêu.

4.2 Khởi tạo nghiệm

VNS bắt đầu từ một tập nghiệm khả thi thay vì một nghiệm duy nhất. Cách xây dựng được thực hiện theo ba ý tưởng chính. Trước hết, chọn một số điểm triển khai không vượt quá số xe tải cho phép; các điểm gần depot được ưu tiên hơn vì chi phí xe tải $2c_{0d}$ nhỏ hơn. Tiếp theo, mỗi cạnh yêu cầu được gán cho một điểm triển khai đã chọn, với xác suất lớn hơn cho các điểm triển khai gần cạnh đó. Cuối cùng, tại mỗi điểm triển khai, các cạnh được nối thành một hành trình lớn rồi tách thành nhiều chuyến bay sao cho mỗi chuyến không vượt quá giới hạn bay L .

Sau khi tạo tập nghiệm ban đầu, mỗi nghiệm được cải thiện nhanh bằng một VND nhẹ. Nghiệm có giá trị $f(S)$ nhỏ nhất được chọn làm nghiệm hiện tại của VNS. Cách này giúp thuật toán tránh phụ thuộc quá mạnh vào một nghiệm khởi tạo ngẫu nhiên, đồng thời vẫn giữ chi phí khởi tạo ở mức thấp.

4.3 Các lân cận sử dụng trong VNS

Các lân cận được thiết kế để bảo toàn tính khả thi: mỗi cạnh yêu cầu được phục vụ đúng một lần, mỗi chuyến bay bắt đầu và kết thúc tại cùng một điểm triển khai, và độ dài chuyến bay không vượt quá L . Sau mỗi thao tác, nghiệm chỉ được chấp nhận nếu khả thi và cải thiện hàm mục tiêu.

4.3.1 Tối ưu bên trong một chuyến bay

Lân cận này giữ nguyên tập cạnh của từng chuyến bay nhưng thay đổi thứ tự phục vụ và hướng đi qua từng cạnh. Mục tiêu là giảm $C(F)$ mà không làm thay đổi phân bố công việc giữa các điểm triển khai. Đây là bước tăng cường cục bộ rẻ và thường được gọi trước các thao tác trao đổi lớn hơn.

Với các chuyến bay nhỏ, bài toán sắp xếp lại thứ tự cạnh có thể được giải chính xác hoặc gần chính xác bằng bộ tối ưu chuyến bay. Với các chuyến bay lớn hơn, ta chỉ thực hiện cải thiện cục bộ để tránh làm tăng thời gian tính toán.

4.3.2 Destroy–repair trên điểm triển khai nghẽn

Destroy–repair loại bỏ một số cạnh khỏi các chuyến bay của điểm triển khai nghẽn $b(S)$, sau đó chèn lại từng cạnh vào vị trí tốt nhất còn khả thi. Vị trí chèn có thể nằm trong một chuyến bay hiện có hoặc tạo thành một chuyến bay mới tại một điểm triển khai đã chọn.

Toán tử này vừa có khả năng giảm tải cho điểm triển khai nghẽn, vừa cho phép tái phân bố một phần công việc sang các điểm triển khai khác. Vì vậy, nó đóng vai trò cầu nối giữa tìm kiếm cục bộ nhỏ và nhiễu loạn lớn trong VNS.

4.3.3 Dịch chuyển chuỗi 0- ℓ

Lân cận 0- ℓ chọn một chuỗi liên tiếp gồm ℓ cạnh trong một chuyến bay của điểm triển khai nghẽn và chuyển chuỗi này sang một chuyến bay khác. Trường hợp đích có thể là một chuyến bay đã tồn tại hoặc một chuyến bay mới. Tham số ℓ tăng dần từ 1 đến $\ell_{\max}$; khi tìm được cải thiện, thuật toán quay lại xét từ $\ell = 1$.

Lân cận này phù hợp với bài toán min-max vì nó trực tiếp rút bớt tải khỏi điểm triển khai đang quyết định giá trị mục tiêu. Tuy nhiên, mọi dịch chuyển đều phải kiểm tra lại giới hạn bay của chuyến nhận.

4.3.4 Trao đổi chuỗi ℓ_1 - ℓ_2

Lân cận ℓ_1 - ℓ_2 hoán đổi một chuỗi ℓ_1 cạnh từ điểm triển khai nghẽn với một chuỗi ℓ_2 cạnh từ chuyến bay khác. So với 0- ℓ , thao tác này cân bằng hơn vì điểm triển khai nghẽn không chỉ chuyển công việc đi mà còn nhận lại một phần công việc khác có thể phù hợp hơn về mặt hình học.

Các giá trị ℓ_1, ℓ_2 được giới hạn bởi $\ell_{\max}$ để kiểm soát độ phức tạp. Khi một cải thiện được tìm thấy, quá trình xét chuỗi được khởi động lại từ kích thước nhỏ nhất.

4.4 VND như thủ tục tăng cường

Trong VNS, Variable Neighborhood Descent (VND) được dùng làm thủ tục tăng cường sau mỗi lần nhiễu loạn. VND xét tuần tự một danh sách lân cận. Nếu một lân cận tạo ra nghiệm tốt hơn, nghiệm hiện tại được cập nhật và quá trình xét lân cận quay lại từ đầu. Nếu không có lân cận nào cải thiện, nghiệm được xem là cực trị cục bộ đối với danh sách lân cận đang xét.

Danh sách VND đầy đủ gồm bốn nhóm thao tác:

- (i) tối ưu bên trong từng chuyến bay;
- (ii) destroy-repair;
- (iii) dịch chuyển chuỗi 0- ℓ ;
- (iv) trao đổi chuỗi ℓ_1 - ℓ_2 .

Sau khi VND đầy đủ dừng, có thể áp dụng thêm một bước tối ưu chính xác cho các chuyến bay thuộc điểm triển khai nghẽn. Nếu bước này cải thiện nghiệm, VND được khởi động lại.

Trong vòng lặp ngoài của VNS, ta dùng một VND nhẹ hơn, chỉ gồm tối ưu bên trong chuyến bay và dịch chuyển 0- ℓ . Lý do là giai đoạn này được gọi lặp lại nhiều lần sau shaking; dùng VND đầy đủ ở mọi vòng có thể làm thuật toán tiêu tốn quá nhiều thời gian cho tăng cường cục bộ. VND đầy đủ được dành cho bước cuối cùng để tinh chỉnh nghiệm tốt nhất tìm được.

4.5 Cơ chế shaking

Shaking là bước tạo nhiều có kiểm soát nhằm đưa nghiệm ra khỏi cực trị cục bộ. Với mức lân cận k , thuật toán áp dụng k lần destroy–repair liên tiếp lên nghiệm hiện tại. Khi k nhỏ, nghiệm mới chỉ khác nhẹ so với nghiệm gốc; khi k tăng, mức nhiễu lớn hơn và khả năng rời khỏi vùng tìm kiếm hiện tại cũng cao hơn.

Cách shaking này phù hợp với cấu trúc MM-MT-dLARP vì nó không phá toàn bộ nghiệm. Các chuyển bay và điểm triển khai đang tốt được giữ lại phần lớn, trong khi một phần công việc tại điểm triển khai nghẽn được phân bố lại. Nhờ đó, shaking tạo đa dạng nhưng vẫn tận dụng được cấu trúc của nghiệm hiện tại.

4.6 Khung thuật toán VNS

Thuật toán VNS cho MM-MT-dLARP có thể mô tả cô đọng như sau.

- Bước 1.** Tạo một tập nghiệm ban đầu khả thi và cải thiện nhanh từng nghiệm bằng VND nhẹ.
- Bước 2.** Chọn nghiệm tốt nhất làm nghiệm hiện tại S và nghiệm tốt nhất toàn cục S^* .
- Bước 3.** Đặt mức shaking $k = 1$.
- Bước 4.** Tạo nghiệm nhiễu S' từ S bằng shaking mức k .
- Bước 5.** Cải thiện S' bằng VND nhẹ, thu được nghiệm $\bar{S}$.
- Bước 6.** Nếu $f(\bar{S}) < f(S)$, cập nhật $S \leftarrow \bar{S}$ và đặt lại $k = 1$. Nếu $\bar{S}$ cũng tốt hơn S^* , cập nhật $S^* \leftarrow \bar{S}$.
- Bước 7.** Nếu không cải thiện, tăng $k \leftarrow k + 1$. Khi $k > k_{\max}$, bắt đầu lại từ $k = 1$ cho vòng lặp ngoài tiếp theo.
- Bước 8.** Khi đạt số vòng lặp hoặc giới hạn thời gian, áp dụng VND đầy đủ lên S^* và trả về nghiệm cuối cùng.

Quy tắc chấp nhận trong chương này là chấp nhận cải thiện nghiêm ngặt. Nghĩa là một nghiệm mới chỉ thay thế nghiệm hiện tại khi giá trị mục tiêu giảm. Quy tắc này đơn giản, ổn định và phù hợp với mục tiêu min–max vì mọi cải thiện đều làm giảm trực tiếp tải lớn nhất trong nghiệm.

4.7 Tham số chính

Các tham số của VNS được chia thành ba nhóm: tham số khởi tạo, tham số lân cận và tham số điều khiển vòng lặp. Bảng 4.1 tóm tắt các tham số quan trọng.

Bảng 4.1: Các tham số chính của VNS

Tham số	Ý nghĩa
$P_{\max}$	Số nghiệm tối đa trong tập nghiệm khởi tạo.
$n_{\max}$	Số cạnh tối đa bị loại trong một lần destroy.
I_{repair}	Số lần thử destroy–repair trước khi dừng toán tử.
$\ell_{\max}$	Độ dài chuỗi lớn nhất trong các lân cận $0-\ell$ và $\ell_1-\ell_2$.
$k_{\max}$	Mức shaking lớn nhất trước khi quay lại mức nhỏ nhất.
$I_{\max}$	Số vòng lặp ngoài tối đa của VNS.
$T_{\max}$	Giới hạn thời gian tính toán, nếu được sử dụng.

Việc chọn tham số phản ánh sự đánh đổi giữa thời gian và chất lượng nghiệm. Giá trị lớn của $k_{\max}$, $\ell_{\max}$ hoặc $I_{\max}$ giúp thuật toán khảo sát không gian nghiệm rộng hơn, nhưng làm tăng thời gian chạy. Trong thực nghiệm, các tham số này nên được giữ vừa phải để VNS có thể thực hiện nhiều vòng shaking và cải thiện thay vì dành quá nhiều thời gian cho một lân cận duy nhất.

4.8 Nhận xét

VNS phù hợp với MM-MT-dLARP vì bài toán có cấu trúc min–max rõ rệt. Thay vì cải thiện tổng chi phí một cách dần trải, thuật toán tập trung vào điểm triển khai đang tạo ra giá trị lớn nhất của hàm mục tiêu. Các lân cận nhỏ giúp tinh chỉnh thứ tự phục vụ trong từng chuyến bay, trong khi các lân cận chuyển và trao đổi chuỗi cho phép cân bằng lại tải giữa các điểm triển khai.

Cơ chế shaking giúp thuật toán không bị kẹt quá lâu tại một cực trị cục bộ. Tuy nhiên, shaking vẫn dựa trên destroy–repair nên nghiệm mới thường giữ được phần lớn cấu trúc tốt của nghiệm cũ. Nhờ sự kết hợp giữa tăng cường và đa dạng hoá, VNS là một lựa chọn tự nhiên cho các bài toán định tuyến drone nhiều chuyến, nơi không gian nghiệm lớn và các ràng buộc khả năng bay làm cho việc tối ưu chính xác trở nên khó khăn.

Chương 5

Giải thuật Large Neighborhood Search cho MM-MT-dLARP

5.1 Vai trò của LNS trong bài toán MM-MT-dLARP

Large Neighborhood Search (LNS) là một khung tìm kiếm cục bộ mở rộng, trong đó nghiệm hiện tại được cải thiện bằng cách phá một phần nghiệm rồi sửa lại phần bị phá [12, 11, 10]. Khác với các lân cận nhỏ chỉ thay đổi một vài cạnh hoặc một vài vị trí trong chuyến bay, LNS cho phép tái cấu trúc một phần lớn của nghiệm. Vì vậy, LNS phù hợp với các bài toán định tuyến có không gian nghiệm lớn và nhiều ràng buộc khả thi.

Với bài toán MM-MT-dLARP, một nghiệm phải quyết định đồng thời tập điểm triển khai drone được chọn, cách phân bố các cạnh yêu cầu cho từng điểm triển khai, và cách chia các cạnh đó thành nhiều chuyến bay khả thi [5]. Tương tự Chương 4, cho một nghiệm S , ký hiệu $D(S)$ là tập điểm triển khai được chọn và $\mathcal{F}_d(S)$ là tập các chuyến bay gắn với điểm triển khai d . Chi phí của một chuyến bay F được ký hiệu bởi $C(F)$. Hàm mục tiêu min-max được viết dưới dạng

$$f(S) = \max_{d \in D(S)} \left(2c_{0d} + \sum_{F \in \mathcal{F}_d(S)} C(F) \right), \quad (5.1)$$

trong đó c_{0d} là khoảng cách từ depot đến điểm triển khai d .

Do mục tiêu là giảm tải lớn nhất trong các điểm triển khai được sử dụng, cải thiện nghiệm thường gắn với việc giảm tải tại điểm triển khai nghẽn. Ta gọi

$$b(S) = \arg \max_{d \in D(S)} \left(2c_{0d} + \sum_{F \in \mathcal{F}_d(S)} C(F) \right) \quad (5.2)$$

là điểm triển khai nghẽn của nghiệm S . Trong LNS, bước phá nghiệm thường ưu tiên tác động lên các chuyến bay thuộc $b(S)$, còn bước sửa nghiệm tìm cách chèn lại các cạnh bị loại sao cho tải giữa các điểm triển khai được cân bằng hơn.

5.2 Khởi tạo nghiệm

LNS bắt đầu từ một tập nghiệm khả thi thay vì một nghiệm duy nhất. Cách xây dựng nghiệm ban đầu tương tự như trong VNS. Trước hết, chọn một số điểm triển khai không vượt quá số xe tải cho phép; các điểm gần depot được ưu tiên hơn vì có chi phí xe tải nhỏ hơn. Tiếp theo, mỗi cạnh yêu cầu được gán cho một điểm triển khai đã chọn, với xác suất lớn hơn cho các điểm triển khai gần cạnh đó. Cuối cùng, tại mỗi điểm triển khai, các cạnh được sắp thành một hành trình lớn rồi tách thành nhiều chuyến bay sao cho mỗi chuyến không vượt quá giới hạn bay L .

Sau khi sinh tập nghiệm ban đầu, mỗi nghiệm được cải thiện nhanh bằng một VND nhẹ. Một số nghiệm tốt nhất được giữ lại làm nghiệm xuất phát cho LNS. Cách này giúp thuật toán khai thác nhiều cấu trúc nghiệm khác nhau, thay vì phụ thuộc hoàn toàn vào một nghiệm khởi tạo ngẫu nhiên.

5.3 Cơ chế destroy–repair trong LNS

Trọng tâm của LNS là cơ chế destroy–repair. Với nghiệm hiện tại S , thuật toán loại bỏ một tập cạnh yêu cầu khỏi nghiệm, sau đó chèn lại các cạnh này vào những vị trí khả thi. Nếu tập cạnh bị loại đủ lớn, nghiệm sau repair có thể khác đáng kể nghiệm ban đầu, từ đó giúp thuật toán thoát khỏi cực trị cục bộ.

5.3.1 Toán tử destroy

Toán tử destroy chọn một tập cạnh Q đang được phục vụ trong nghiệm và loại chúng khỏi các chuyến bay hiện tại. Kích thước của Q được xác định bởi tỷ lệ phá ρ :

$$|Q| = \max \{1, \lfloor \rho |E_R| \rfloor \}, \quad (5.3)$$

trong đó E_R là tập các cạnh yêu cầu.

Có hai cách chọn cạnh bị loại được sử dụng. Cách thứ nhất là chọn ngẫu nhiên các cạnh trong nghiệm. Cách này tạo đa dạng và giúp thuật toán không bị phụ thuộc quá mạnh vào cấu trúc hiện tại. Cách thứ hai là chọn ưu tiên các cạnh thuộc điểm triển khai ngẫu nhiên $b(S)$. Cách này phù hợp với mục tiêu min–max vì nó trực tiếp làm giảm tải tại điểm triển khai đang quyết định giá trị hàm mục tiêu.

Sau khi loại bỏ Q , các chuyến bay còn lại vẫn giữ nguyên thứ tự tương đối. Nghiệm tạm thời có thể chưa phục vụ đầy đủ tất cả các cạnh yêu cầu, nhưng vẫn giữ được phần lớn cấu trúc tốt của nghiệm cũ. Tính đầy đủ của nghiệm sẽ được khôi phục trong bước repair.

5.3.2 Toán tử repair

Toán tử repair lần lượt chèn lại các cạnh trong Q . Với một cạnh $e \in Q$, thuật toán xét các khả năng chèn e vào một chuyến bay hiện có, chèn theo một trong hai hướng phục

vụ, hoặc tạo một chuyến bay mới tại một điểm triển khai đã được chọn. Mỗi phương án chỉ được chấp nhận nếu chuyến bay sau khi chen vẫn thỏa mãn giới hạn bay L .

Trong số các phương án khả thi, thuật toán chọn phương án làm giảm tốt nhất giá trị mục tiêu sau khi chen. Nếu p là vị trí chen, o là hướng phục vụ của cạnh, và $S \oplus (e, d, F, p, o)$ là nghiệm thu được sau khi chen e vào chuyến bay $F \in \mathcal{F}_d(S)$, phương án được chọn là

$$(d^*, F^*, p^*, o^*) = \underset{(d, F, p, o)}{\operatorname{argmin}} f(S \oplus (e, d, F, p, o)). \quad (5.4)$$

Quy tắc chen này mang tính tham lam vì mỗi cạnh được đặt vào vị trí tốt nhất tại thời điểm xét. Tuy nhiên, do nhiều cạnh đã bị loại cùng lúc, nghiệm sau repair vẫn có thể thay đổi đáng kể so với nghiệm trước destroy. Đây là điểm khác biệt chính giữa LNS và các lân cận nhỏ trong tìm kiếm cục bộ.

5.4 Tăng cường cục bộ sau repair

Sau khi tất cả các cạnh bị loại được chen lại, nghiệm thu được tiếp tục được cải thiện bằng một VND nhẹ. Trong vòng lặp chính của LNS, VND nhẹ chỉ gồm các thao tác có chi phí thấp, chẳng hạn tối ưu thứ tự phục vụ bên trong từng chuyến bay và dịch chuyển một số cạnh ngăn giữa các chuyến bay. Mục tiêu của bước này là loại bỏ các sai lệch rõ ràng do repair tham lam tạo ra mà không làm tăng quá nhiều thời gian tính toán.

Ở cuối quá trình LNS, nghiệm tốt nhất được cải thiện thêm bằng VND đầy đủ. VND đầy đủ xét lại các nhóm lân cận đã trình bày ở Chương 3, bao gồm tối ưu bên trong chuyến bay, destroy–repair, dịch chuyển chuỗi 0 – ℓ , và trao đổi chuỗi ℓ_1 – ℓ_2 . Cách tổ chức này giúp LNS vừa có khả năng tạo bước nhảy lớn trong không gian nghiệm, vừa khai thác sâu quanh các nghiệm hứa hẹn.

5.5 Quy tắc chấp nhận và cập nhật nghiệm

Trong chương này, LNS sử dụng quy tắc chấp nhận cải thiện nghiệm ngặt. Nghĩa là nghiệm mới S' chỉ thay thế nghiệm hiện tại S nếu

$$f(S') < f(S). \quad (5.5)$$

Quy tắc này đơn giản, ổn định và đồng bộ với cách chấp nhận nghiệm trong VNS. Với mục tiêu min–max, mỗi lần chấp nhận đều tương ứng với việc giảm trực tiếp tải lớn nhất trong nghiệm.

Nghiệm tốt nhất toàn cục S^* được cập nhật độc lập với nghiệm hiện tại. Nếu nghiệm sau repair và tăng cường cục bộ có giá trị tốt hơn S^* , thuật toán cập nhật $S^* \leftarrow S'$. Sau khi đạt giới hạn vòng lặp hoặc giới hạn thời gian, S^* được dùng làm nghiệm đầu vào cho bước tinh chỉnh cuối cùng.

5.6 Pha chia nhỏ điểm phục vụ

Trong MM-MT-dLARP, các điểm phục vụ trên cạnh yêu cầu được xét thông qua một quá trình rời rạc hoá. Do đó, chất lượng nghiệm phụ thuộc không chỉ vào cách phân công cạnh cho điểm triển khai, mà còn vào mức chi tiết của tập điểm rời rạc đang được sử dụng.

Sau khi LNS tìm được các nghiệm tốt trên thể hiện rời rạc ban đầu, thuật toán áp dụng một pha chia nhỏ. Các điểm mới, thường là trung điểm của các khoảng hiện có trên cạnh yêu cầu, được bổ sung để tạo một thể hiện tinh hơn. Nghiệm tốt nhất hiện tại được chuyển sang thể hiện mới và tiếp tục được cải thiện bằng LNS hoặc VND.

Quá trình chia nhỏ có thể được thực hiện thích nghi. Thay vì làm mịn toàn bộ mạng lưới, thuật toán chỉ tiếp tục chia nhỏ các vùng có điểm mới được sử dụng trong nghiệm tốt. Nhờ đó, không gian nghiệm được mở rộng quanh những vùng có tiềm năng cải thiện, trong khi chi phí tính toán vẫn được kiểm soát. Cơ chế này kế thừa tinh thần của thuật toán trong [5] và kết hợp tự nhiên với LNS.

5.7 Khung thuật toán LNS

Thuật toán LNS cho MM-MT-dLARP có thể mô tả cô đọng như sau.

- Bước 1.** Tạo một tập nghiệm ban đầu khả thi và cải thiện nhanh từng nghiệm bằng VND nhẹ.
- Bước 2.** Chọn một số nghiệm tốt nhất làm nghiệm xuất phát cho LNS.
- Bước 3.** Với mỗi nghiệm xuất phát, đặt nghiệm hiện tại là S và nghiệm tốt nhất toàn cục là S^* .
- Bước 4.** Chọn một tập cạnh Q để loại khỏi nghiệm bằng toán tử destroy.
- Bước 5.** Chèn lại các cạnh trong Q bằng toán tử repair, thu được nghiệm S' .
- Bước 6.** Cải thiện S' bằng VND nhẹ, thu được nghiệm $\bar{S}$.
- Bước 7.** Nếu $f(\bar{S}) < f(S)$, cập nhật $S \leftarrow \bar{S}$. Nếu $\bar{S}$ cũng tốt hơn S^* , cập nhật $S^* \leftarrow \bar{S}$.
- Bước 8.** Lặp lại các bước destroy, repair và tăng cường cục bộ cho đến khi đạt số vòng lặp hoặc giới hạn thời gian.
- Bước 9.** Áp dụng VND đầy đủ lên các nghiệm tốt nhất sau LNS.
- Bước 10.** Thực hiện pha chia nhỏ điểm phục vụ cho một số nghiệm tốt nhất.
- Bước 11.** Trả về nghiệm có giá trị hàm mục tiêu nhỏ nhất.

5.8 Tham số chính

Các tham số của LNS được chia thành ba nhóm: tham số khởi tạo, tham số destroy-repair và tham số điều khiển vòng lặp. Bảng 5.1 tóm tắt các tham số quan trọng.

Bảng 5.1: Các tham số chính của LNS

Tham số	Ý nghĩa
$P_{\max}$	Số nghiệm tối đa trong tập nghiệm khởi tạo.
ρ	Tỷ lệ cạnh bị loại trong một lần destroy.
$n_{\max}$	Số cạnh tối đa bị loại trong một lần destroy, nếu dùng giới hạn tuyệt đối.
I_{repair}	Số lần thử repair hoặc số phương án chèn được xét trước khi dừng toán tử.
$I_{\max}$	Số vòng lặp chính tối đa của LNS.
$T_{\max}$	Giới hạn thời gian tính toán, nếu được sử dụng.
P_{split}	Số nghiệm tốt nhất được đưa vào pha chia nhỏ điểm phục vụ.

Tỷ lệ phá ρ là tham số quan trọng nhất của LNS. Nếu ρ quá nhỏ, thuật toán gần với tìm kiếm cục bộ thông thường và khó thoát khỏi cực trị cục bộ. Nếu ρ quá lớn, bước repair trở nên tốn kém và có thể phá vỡ quá nhiều cấu trúc tốt của nghiệm hiện tại. Vì vậy, ρ thường được chọn ở mức trung bình để cân bằng giữa khai thác và khám phá.

Số nghiệm được đưa vào pha chia nhỏ cũng ảnh hưởng đến chất lượng và thời gian chạy. Nếu chỉ chia nhỏ nghiệm tốt nhất, thuật toán nhanh hơn nhưng dễ phụ thuộc vào một cấu trúc nghiệm duy nhất. Nếu chia nhỏ nhiều nghiệm tốt, thuật toán có cơ hội phát hiện cấu trúc tốt hơn trên thể hiện tình hơn, đổi lại chi phí tính toán cao hơn.

5.9 Nhận xét

LNS phù hợp với MM-MT-dLARP vì bài toán có cấu trúc phân công phức tạp giữa điểm triển khai, chuyển bay và cạnh yêu cầu. Các lân cận nhỏ thường chỉ cải thiện được thứ tự phục vụ hoặc trao đổi một số ít cạnh, trong khi LNS có thể tái cấu trúc đồng thời nhiều chuyển bay. Điều này đặc biệt quan trọng với mục tiêu min-max, nơi một cải thiện tốt thường đến từ việc chuyển tải ra khỏi điểm triển khai nghẽn thay vì chỉ giảm tổng chi phí cục bộ.

So với VNS, LNS sử dụng lân cận lớn hơn và phụ thuộc nhiều hơn vào chất lượng của bước repair. Nếu repair đủ tốt, thuật toán có thể tạo ra các bước nhảy mạnh trong không gian nghiệm mà vẫn duy trì tính khả thi. Ngược lại, nếu repair quá tham lam, nghiệm thu được có thể hợp lệ nhưng chưa cân bằng tốt. Vì vậy, trong thiết kế được sử dụng, LNS

được kết hợp với VND nhẹ sau mỗi lần repair, VND đầy đủ ở cuối quá trình, và pha chia nhỏ điểm phục vụ cho các nghiệm hứa hẹn. Sự kết hợp này giúp thuật toán vừa đa dạng hoá vùng tìm kiếm, vừa khai thác sâu quanh các cấu trúc nghiệm tốt.

Chương 6

Thực nghiệm và đánh giá kết quả

6.1 Mục tiêu thực nghiệm

Các chương trước đã trình bày cách biểu diễn nghiệm và thiết kế ba hướng tìm kiếm cục bộ cho bài toán MM-MT-dLARP: VND, VNS và LNS. Chương này trình bày phần thực nghiệm nhằm so sánh ba giải thuật trên cùng một tập cấu hình đầu vào. Mục tiêu chính của thực nghiệm không phải là chứng minh tối ưu toàn cục, mà là đánh giá tương đối chất lượng nghiệm và thời gian chạy của từng giải thuật trong cùng điều kiện cài đặt.

Ba giải thuật được so sánh gồm:

- **VND**: thuật toán descent dùng một danh sách lân cận cố định và dừng khi không còn lân cận nào cải thiện nghiệm;
- **VNS**: thuật toán mở rộng VND bằng cơ chế shaking để thoát khỏi cực trị cục bộ;
- **LNS**: thuật toán phá-sửa nghiệm trên lân cận lớn, sau đó cải thiện lại bằng tìm kiếm cục bộ.

Tiêu chí đánh giá chính là giá trị hàm mục tiêu min-max, tương ứng với makespan lớn nhất giữa các xe tải-drone. Giá trị này càng nhỏ thì nghiệm càng tốt. Ngoài ra, thời gian chạy cũng được ghi nhận để đánh giá chi phí tính toán của từng giải thuật.

6.2 Thiết lập thực nghiệm

Các thể hiện dùng trong thực nghiệm được lấy từ bộ dữ liệu của bài báo *The Min-Max Multi-Trip Drone Location Arc Routing Problem* [5]. Đây là các thể hiện rời rạc của bài toán MM-MT-LARP, được lưu trong thư mục `data/instances`. Toàn bộ thư mục dữ liệu hiện có 136 file `.dat`, được chia theo các họ MTLARP4, MTLARP5, MTLARP6, MTLARP7, MTLARP8, MTLARP9 và MTLARP10. Tên file có dạng `MTLARP $q$ _ $n$ _ $p$ _ $r$ .dat`, trong đó tiền tố MTLARP q biểu diễn nhóm thể hiện, các chỉ số tiếp theo mô tả kích thước hoặc cấu hình sinh dữ liệu, còn chỉ số cuối là số thứ tự lặp của thể hiện trong cùng nhóm.

Trong khuôn khổ đề án, thực nghiệm tập trung trên hai nhóm thể hiện nhỏ và trung bình là MTLARP4 và MTLARP5. Việc chọn hai nhóm này nhằm bảo đảm cả ba thuật toán VND, VNS và LNS đều có thể chạy trên cùng một tập cấu hình trong thời gian hợp lý, đồng thời vẫn thể hiện được sự khác biệt về chất lượng nghiệm khi kích thước bài toán tăng. Với mỗi thể hiện, tham số P biểu diễn số xe tải-drone được sử dụng, còn L là giới hạn chi phí tối đa của mỗi chuyến bay drone. Giá trị P và L được chọn theo cấu hình thực nghiệm tương ứng với bộ test gốc của bài báo, sau đó được dùng thống nhất cho cả ba thuật toán.

Mỗi cấu hình (Thể hiện, P, L) được chạy lần lượt với ba giải thuật VNS, LNS và VND. Các lượt chạy sử dụng cùng seed ngẫu nhiên bằng 0, cùng kiểu mục tiêu `minmax_ghg`, cùng bộ tối ưu chuyến bay `bc`, và cùng chế độ tắt log chi tiết để giảm ảnh hưởng của xuất nhập chuẩn đến thời gian chạy. Kết quả đầu ra được lưu trong hai thư mục:

- `exp/results/vns_lns_vnd_configs/results.xlsx`;
- `exp/results/vns_lns_vnd_configs_mtl5/results.xlsx`.

Mỗi file kết quả cũng có bản `csv` tương ứng để thuận tiện cho việc tổng hợp số liệu. Các cột quan trọng trong file kết quả gồm tên giải thuật, tên thể hiện, số xe tải P , giới hạn bay L , số xe tải được dùng, giá trị mục tiêu, makespan theo cách tính của bài báo, makespan theo mục tiêu phát thải, tổng chi phí phát thải, thời gian chạy và trạng thái thực thi. Trong toàn bộ thực nghiệm, các lượt chạy đều có trạng thái `ok`, tức là không có lượt chạy bị lỗi.

6.3 Tập cấu hình thực nghiệm

Bảng 6.1 liệt kê các cấu hình thuộc nhóm MTLARP4. Nhóm này gồm 20 cấu hình, tương ứng với 60 lượt chạy khi xét ba giải thuật.

Bảng 6.1: Các cấu hình thực nghiệm nhóm MTLARP4

Thể hiện	P	L
MTLARP4_4_3_1.dat	2	847
MTLARP4_4_3_2.dat	2	1033
MTLARP4_4_4_1.dat	2	746
MTLARP4_4_4_1.dat	3	746
MTLARP4_4_4_2.dat	2	847
MTLARP4_4_4_2.dat	3	847
MTLARP4_4_5_1.dat	2	785
MTLARP4_4_5_1.dat	3	785
MTLARP4_4_5_1.dat	4	785
MTLARP4_4_5_2.dat	2	900

Thể hiện	P	L
MTLARP4_4_5_2.dat	3	900
MTLARP4_4_5_2.dat	4	900
MTLARP4_4_6_1.dat	2	694
MTLARP4_4_6_1.dat	3	694
MTLARP4_4_6_1.dat	4	694
MTLARP4_4_6_1.dat	5	694
MTLARP4_4_6_2.dat	2	903
MTLARP4_4_6_2.dat	3	903
MTLARP4_4_6_2.dat	4	903
MTLARP4_4_6_2.dat	5	903

Bảng 6.2 liệt kê các cấu hình thuộc nhóm MTLARP5. Nhóm này cũng gồm 20 cấu hình, tương ứng với 60 lượt chạy.

Bảng 6.2: Các cấu hình thực nghiệm nhóm MTLARP5

Thể hiện	P	L
MTLARP5_5_3_1.dat	2	1469
MTLARP5_5_3_2.dat	2	1368
MTLARP5_5_4_1.dat	2	1159
MTLARP5_5_4_1.dat	3	1159
MTLARP5_5_4_2.dat	2	1010
MTLARP5_5_4_2.dat	3	1010
MTLARP5_5_5_1.dat	2	955
MTLARP5_5_5_1.dat	3	955
MTLARP5_5_5_1.dat	4	955
MTLARP5_5_5_2.dat	2	988
MTLARP5_5_5_2.dat	3	988
MTLARP5_5_5_2.dat	4	988
MTLARP5_5_6_1.dat	2	783
MTLARP5_5_6_1.dat	3	783
MTLARP5_5_6_1.dat	4	783
MTLARP5_5_6_1.dat	5	783
MTLARP5_5_6_2.dat	2	1168
MTLARP5_5_6_2.dat	3	1168
MTLARP5_5_6_2.dat	4	1168
MTLARP5_5_6_2.dat	5	1168

Tổng cộng hai nhóm dữ liệu có 40 cấu hình và 120 lượt chạy. Việc sử dụng cùng một

tập cấu hình cho cả ba giải thuật giúp kết quả so sánh phản ánh trực tiếp sự khác biệt giữa các cơ chế tìm kiếm.

6.4 Cấu hình tham số của các thuật toán

Bảng 6.3 trình bày các tham số chính được sử dụng trong các lượt chạy thực nghiệm. Các tham số này được giữ cố định trên toàn bộ 40 cấu hình để bảo đảm kết quả so sánh không bị ảnh hưởng bởi việc tinh chỉnh riêng cho từng thể hiện.

Bảng 6.3: Cấu hình tham số của các thuật toán trong thực nghiệm

Thành phần	Tham số	Giá trị sử dụng
Cấu hình chung	Seed	0
Cấu hình chung	Kiểu mục tiêu	minmax_ghg
Cấu hình chung	Bộ tối ưu chuyển bay	bc
Cấu hình chung	Số nghiệm xây dựng tối đa	20
Cấu hình chung	Số nghiệm xét ở pha splitting	10
Cấu hình chung	Ngưỡng tối ưu exact cho chuyển bay	12 cạnh yêu cầu
VND	Danh sách lân cận	intraroute_move, destroy_and_repair, zero_to_l_exchange, l1_to_l2_exchange
VND	nmax_destroy	5
VND	itmax_destroy	30
VND	lmax_exchange	3
VNS	k_max	4
VNS	max_iter	100
VNS	Tìm kiếm cục bộ trong vòng lặp	intraroute_move, zero_to_l_exchange
LNS	destroy_frac	0.3
LNS	max_iter	500
LNS	Quy tắc chấp nhận	Chỉ chấp nhận nghiệm cải thiện
LNS	Tìm kiếm cục bộ sau repair	intraroute_move

Với VND, các tham số `nmax_destroy`, `itmax_destroy` và `lmax_exchange` lần lượt điều khiển kích thước phá-sửa, số lần thử trong toán tử phá-sửa và độ dài tối đa của chuỗi

cạnh được xét trong các phép trao đổi. Với VNS, tham số $k_{\max}=4$ giới hạn mức shaking lớn nhất trước khi quay lại lân cận đầu tiên, còn $\max_iter=100$ là số vòng lặp ngoài. Với LNS, $\text{destroy_frac}=0.3$ nghĩa là mỗi bước destroy loại bỏ khoảng 30% số nhiệm vụ hiện có rồi chèn lại bằng chiến lược best insertion; thuật toán chạy tối đa 500 vòng lặp ngoài cho mỗi nghiệm xuất phát được chọn.

Các cấu hình trên phản ánh chủ đích thực nghiệm của đề án: VND đóng vai trò baseline tìm kiếm cục bộ thuần túy; VNS bổ sung cơ chế shaking nhưng vẫn giữ chi phí tính toán ở mức vừa phải; còn LNS sử dụng lân cận lớn hơn, thời gian chạy dài hơn và được kỳ vọng cải thiện tốt hơn các nghiệm bị nghẽn bởi phân công ban đầu.

6.5 Cách tổng hợp kết quả

Với mỗi cấu hình, gọi z_a là giá trị mục tiêu thu được bởi giải thuật $a \in \{\text{VNS}, \text{LNS}, \text{VND}\}$. Giá trị tốt nhất trong ba giải thuật trên cùng cấu hình được tính bởi

$$z_{\min} = \min_a z_a. \quad (6.1)$$

Độ lệch tương đối của giải thuật a so với nghiệm tốt nhất trong cùng cấu hình được tính theo công thức

$$\text{gap}_a = \frac{z_a - z_{\min}}{z_{\min}} \times 100\%. \quad (6.2)$$

Nếu một giải thuật đạt đúng giá trị $z_{\min}$ trên một cấu hình, giải thuật đó được tính là thắng trên cấu hình đó. Trong trường hợp nhiều giải thuật đạt cùng giá trị tốt nhất, tất cả các giải thuật đó đều được tính thắng. Vì vậy, tổng số lượt thắng có thể lớn hơn số cấu hình.

6.6 Kết quả tổng hợp

Bảng 6.4 trình bày kết quả tổng hợp trên nhóm MTLARP4. Có thể thấy LNS đạt số lượt thắng cao nhất với 17 trên 20 cấu hình. VND đạt nghiệm tốt nhất trên 12 cấu hình, trong khi VNS chỉ thắng trên 1 cấu hình. Xét theo độ lệch trung bình, LNS có gap trung bình thấp nhất, chỉ 0.240%, tiếp theo là VND với 1.304%, còn VNS có gap trung bình 8.122%.

Bảng 6.4: Kết quả tổng hợp trên nhóm MTLARP4

Giải thuật	Số thắng	Mục tiêu TB	Gap TB (%)	Thời gian TB (s)
VNS	1	1530.355	8.122	52.884
LNS	17	1419.480	0.240	23.884
VND	12	1431.224	1.304	12.841

Bảng 6.5 trình bày kết quả tổng hợp trên nhóm MTLARP5. Nhóm này khó hơn nhóm MTLARP4, thể hiện qua giá trị mục tiêu và thời gian chạy trung bình cao hơn. Tuy nhiên, xu hướng so sánh giữa các giải thuật vẫn tương tự: LNS có chất lượng nghiệm tốt nhất với 18 lượt thắng và gap trung bình 0.245%; VND đứng thứ hai với 9 lượt thắng và gap trung bình 1.476%; VNS có chất lượng thấp hơn rõ rệt với gap trung bình 11.262%.

Bảng 6.5: Kết quả tổng hợp trên nhóm MTLARP5

Giải thuật	Số thắng	Mục tiêu TB	Gap TB (%)	Thời gian TB (s)
VNS	1	2684.538	11.262	212.516
LNS	18	2444.463	0.245	132.119
VND	9	2474.274	1.476	79.736

Bảng 6.6 tổng hợp kết quả trên toàn bộ 40 cấu hình. LNS tiếp tục là giải thuật có chất lượng nghiệm tốt nhất với 35 lượt thắng và gap trung bình 0.243%. VND đạt 21 lượt thắng và gap trung bình 1.390%. VNS chỉ đạt 2 lượt thắng và gap trung bình 9.692%.

Bảng 6.6: Kết quả tổng hợp trên toàn bộ thực nghiệm

Giải thuật	Số thắng	Mục tiêu TB	Gap TB (%)	Thời gian TB (s)
VNS	2	2107.446	9.692	132.700
LNS	35	1931.971	0.243	78.002
VND	21	1952.749	1.390	46.289

6.7 Phân tích kết quả

Kết quả thực nghiệm cho thấy LNS là giải thuật ổn định nhất trong ba phương pháp được so sánh. Trên cả hai nhóm thể hiện, LNS có gap trung bình rất nhỏ so với nghiệm tốt nhất tìm được trong từng cấu hình. Điều này phù hợp với đặc trưng của bài toán MM-MT-dLARP: một nghiệm tốt thường cần tái phân bố đồng thời nhiều cạnh giữa các điểm triển khai và các chuyến bay. Các lân cận nhỏ có thể cải thiện thứ tự phục vụ trong từng chuyến bay, nhưng khó thay đổi mạnh cấu trúc phân công. Cơ chế destroy-repair của LNS khắc phục điểm này bằng cách tạm thời loại bỏ một phần nhiệm vụ rồi chèn lại vào các vị trí tốt hơn.

VND có thời gian chạy thấp nhất trong cả hai nhóm dữ liệu. Trên toàn bộ thực nghiệm, thời gian trung bình của VND là 46.289 giây, thấp hơn LNS và VNS. Chất lượng nghiệm của VND cũng khá tốt, với gap trung bình 1.390%. Điều này cho thấy các toán tử tìm kiếm cục bộ được thiết kế trong Chương 3 có hiệu quả ngay cả khi không có cơ chế nhiễu mạnh. Tuy nhiên, do VND là phương pháp descent thuần túy, thuật toán dễ dừng tại cực

trị cục bộ. Vì vậy, VND phù hợp khi cần nghiệm tốt trong thời gian ngắn, nhưng chưa phải lựa chọn tốt nhất nếu ưu tiên chất lượng nghiệm cuối cùng.

VNS có kết quả kém hơn kỳ vọng trong bộ thực nghiệm này. Mặc dù VNS được thiết kế để thoát khỏi cực trị cục bộ bằng shaking, số lượt thắng và gap trung bình đều thấp hơn LNS và VND. Một nguyên nhân có thể là cơ chế shaking hiện tại chưa tạo được sự tái cấu trúc đủ hiệu quả so với destroy-repair của LNS. Nếu shaking chỉ tạo nhiễu quanh nghiệm hiện tại nhưng sau đó VND nhẹ đưa nghiệm quay về một vùng lân cận tương tự, thuật toán sẽ tốn nhiều thời gian mà không cải thiện đáng kể chất lượng nghiệm. Ngoài ra, thời gian chạy trung bình của VNS cao hơn VND nhưng chất lượng nghiệm lại thấp hơn, cho thấy cần tinh chỉnh thêm tham số và chiến lược chấp nhận của VNS.

Khi so sánh hai nhóm thể hiện, nhóm **MTLARP5** có thời gian chạy lớn hơn đáng kể so với nhóm **MTLARP4**. Điều này phù hợp với phân tích độ khó ở Chương 2: khi kích thước đồ thị và số cạnh yêu cầu tăng, số phương án chèn, dịch chuyển và hoán đổi cũng tăng nhanh. Trên nhóm **MTLARP5**, thời gian trung bình của LNS là 132.119 giây, trong khi trên nhóm **MTLARP4** là 23.884 giây. Tuy vậy, gap trung bình của LNS gần như không đổi giữa hai nhóm, lần lượt là 0.240% và 0.245%. Đây là dấu hiệu cho thấy LNS giữ được chất lượng nghiệm ổn định khi kích thước thể hiện tăng trong phạm vi thực nghiệm.

Một điểm cần lưu ý là số lượt thắng của VND và LNS có thể cộng lại lớn hơn số cấu hình do có các trường hợp hòa. Điều này cho thấy trong nhiều cấu hình, VND đã tìm được nghiệm tốt ngang với LNS nhưng với thời gian thấp hơn. Tuy nhiên, LNS thắng trên nhiều cấu hình hơn và có gap trung bình nhỏ hơn, nên nếu xét chất lượng nghiệm tổng thể thì LNS vẫn là phương pháp tốt nhất trong thực nghiệm này.

6.8 Nhận xét chung

Từ các kết quả trên, có thể rút ra ba nhận xét chính. Thứ nhất, LNS là giải thuật cho chất lượng nghiệm tốt nhất và ổn định nhất trên cả hai nhóm dữ liệu. Cơ chế lân cận lớn phù hợp với bản chất phân công và cân bằng tải của MM-MT-dLARP.

Thứ hai, VND là phương pháp có thời gian chạy thấp nhất và vẫn đạt chất lượng nghiệm khá tốt. Điều này cho thấy việc thiết kế đúng các lân cận tập trung vào điểm triển khai nghẽn có vai trò quan trọng. VND có thể được dùng như một thuật toán độc lập khi giới hạn thời gian chặt, hoặc như thủ tục tăng cường bên trong LNS và VNS.

Thứ ba, phiên bản VNS hiện tại chưa khai thác tốt ưu thế của cơ chế thay đổi lân cận. Kết quả này không phủ nhận tính phù hợp của VNS đối với bài toán, nhưng cho thấy cần cải tiến thêm cơ chế shaking, thứ tự lân cận, tham số $k_{\max}$, cũng như quy tắc chấp nhận nghiệm. Các hướng này sẽ được đề cập lại trong Chương 7.

Như vậy, trong phạm vi thực nghiệm của đồ án, LNS là lựa chọn tốt nhất khi ưu tiên chất lượng nghiệm; VND là lựa chọn phù hợp khi ưu tiên thời gian chạy; còn VNS cần được tinh chỉnh thêm để phát huy hiệu quả trên bài toán MM-MT-dLARP.

Chương 7

Kết luận và hướng phát triển

7.1 Kết luận chung

Đồ án đã nghiên cứu bài toán *Min-Max Multi-Trip Drone Location Arc Routing Problem* (MM-MT-dLARP), một biến thể của bài toán định tuyến trên cạnh có xét đồng thời lựa chọn điểm triển khai drone, phân công các cạnh yêu cầu, chia nhiệm vụ thành nhiều chuyến bay và cân bằng tải theo mục tiêu min-max. Đây là một bài toán khó vì kết hợp nhiều tầng quyết định: chọn vị trí, định tuyến trên cạnh, giới hạn thời lượng bay và cân bằng thời gian hoàn thành giữa các xe tải-drone.

Về mặt mô hình, đồ án đã trình bày lại bối cảnh của bài toán, các thành phần chính của MM-MT-dLARP và phiên bản rời rạc MM-MT-LARP. Việc rời rạc hóa giúp chuyển bài toán từ miền liên tục sang một bài toán tối ưu tổ hợp trên đồ thị, trong đó các đoạn tuyến cần phục vụ trở thành các cạnh yêu cầu, còn các đoạn bay không phục vụ được biểu diễn bởi các cạnh bay chết. Trên cơ sở đó, nghiệm được biểu diễn bằng tập điểm triển khai được chọn và danh sách các chuyến bay tại từng điểm triển khai.

Về mặt thuật toán, đồ án tập trung vào ba hướng tìm kiếm cục bộ và metaheuristic: VND, VNS và LNS. VND đóng vai trò là thủ tục cải thiện cơ bản, sử dụng nhiều lân cận như tối ưu trong chuyến bay, phá-sửa, dịch chuyển chuỗi và trao đổi chuỗi. VNS mở rộng VND bằng cơ chế shaking nhằm thoát khỏi cực trị cục bộ. LNS sử dụng lân cận lớn thông qua cơ chế destroy-repair, cho phép tái cấu trúc một phần đáng kể của nghiệm trước khi cải thiện lại bằng tìm kiếm cục bộ.

Phần thực nghiệm đã so sánh ba giải thuật trên hai nhóm dữ liệu MTLARP4 và MTLARP5, với tổng cộng 40 cấu hình và 120 lượt chạy. Kết quả cho thấy LNS là phương pháp có chất lượng nghiệm tốt nhất trong phạm vi thực nghiệm: LNS đạt 35 lượt thắng trên 40 cấu hình và gap trung bình 0.243% so với nghiệm tốt nhất tìm được trong từng cấu hình. VND có thời gian chạy thấp nhất và vẫn đạt chất lượng nghiệm khá tốt, với gap trung bình 1.390%. VNS trong phiên bản hiện tại cho kết quả kém hơn hai phương pháp còn lại, với gap trung bình 9.692%, cho thấy cơ chế shaking và chiến lược điều khiển lân cận cần được cải tiến thêm.

Từ các kết quả trên, có thể kết luận rằng cách tiếp cận tìm kiếm cục bộ là phù hợp

với MM-MT-dLARP. Đặc biệt, các toán tử tập trung vào điểm triển khai nghìn và cơ chế tái phân bổ tải giữa các điểm triển khai có vai trò quan trọng đối với mục tiêu min-max. Trong các phương pháp đã cài đặt, LNS là lựa chọn tốt nhất khi ưu tiên chất lượng nghiệm, còn VND là lựa chọn hợp lý khi cần nghiệm tốt trong thời gian ngắn.

7.2 Các kết quả đạt được

Các kết quả chính của đề án có thể tóm tắt như sau.

- Đề án đã hệ thống hóa bài toán MM-MT-dLARP, bao gồm bối cảnh ứng dụng, mô hình rời rạc, hàm mục tiêu min-max và các ràng buộc quan trọng như phục vụ toàn bộ cạnh yêu cầu, giới hạn thời lượng bay và giới hạn số điểm triển khai.
- Đề án đã xây dựng cách biểu diễn nghiệm phù hợp cho các giải thuật tìm kiếm cục bộ. Nghiệm được mô tả bởi tập điểm triển khai và các chuyến bay gắn với từng điểm triển khai, trong đó mỗi chuyến bay là một chuỗi có thứ tự các cạnh yêu cầu có hướng.
- Đề án đã trình bày các toán tử cải thiện nghiệm, bao gồm tối ưu trong một chuyến bay, destroy-repair, dịch chuyển chuỗi $0-l$, trao đổi chuỗi l_1-l_2 , và tối ưu cục bộ tại điểm triển khai nghìn. Các toán tử này được thiết kế để bảo toàn tính khả thi và ưu tiên giảm tải tại thành phần đang quyết định giá trị mục tiêu.
- Đề án đã xây dựng và so sánh ba khung giải thuật VND, VNS và LNS. Ba giải thuật sử dụng cùng cách biểu diễn nghiệm và cùng nhóm toán tử cơ bản, nhưng khác nhau ở mức độ đa dạng hóa vùng tìm kiếm.
- Đề án đã thực hiện thực nghiệm trên hai nhóm dữ liệu với 40 cấu hình, xuất kết quả ra file Excel và tổng hợp các chỉ số so sánh gồm giá trị mục tiêu trung bình, gap trung bình, số lượt thắng và thời gian chạy trung bình.
- Kết quả thực nghiệm chỉ ra rằng LNS có hiệu quả tốt nhất về chất lượng nghiệm, VND có ưu thế về thời gian chạy, còn VNS cần được điều chỉnh thêm để đạt hiệu quả ổn định hơn.

7.3 Hạn chế

Mặc dù đề án đã đạt được các kết quả thực nghiệm ban đầu, vẫn còn một số hạn chế cần lưu ý.

Thứ nhất, các thực nghiệm mới tập trung vào hai nhóm thể hiện MTLARP4 và MTLARP5. Số lượng cấu hình đủ để quan sát xu hướng so sánh giữa các giải thuật, nhưng chưa đủ rộng để khẳng định hiệu quả trên mọi kích thước và mọi đặc điểm hình học của bài toán.

Các thể hiện lớn hơn hoặc có cấu trúc phân bố cạnh yêu cầu khác có thể làm thay đổi tương quan giữa các giải thuật.

Thứ hai, mỗi cấu hình trong thực nghiệm hiện được chạy với một seed cố định. Do các giải thuật có yếu tố ngẫu nhiên trong khởi tạo, shaking hoặc destroy, kết quả có thể thay đổi khi dùng seed khác. Vì vậy, để đánh giá chắc chắn hơn, cần chạy nhiều lần độc lập cho mỗi cấu hình và báo cáo thêm các chỉ số thống kê như trung bình, độ lệch chuẩn, nghiệm tốt nhất và nghiệm xấu nhất.

Thứ ba, phiên bản VNS hiện tại chưa đạt hiệu quả tốt. Điều này có thể đến từ cách thiết kế shaking, thứ tự lân cận, số vòng lặp hoặc quy tắc chấp nhận nghiệm. Trong cài đặt hiện tại, VNS có thể tiêu tốn nhiều thời gian cho các bước nhiễu và cải thiện nhưng không tạo ra thay đổi đủ mạnh trong cấu trúc phân công nhiệm vụ.

Thứ tư, phần tinh chỉnh tham số còn hạn chế. Các tham số như tỷ lệ phá ρ của LNS, mức shaking $k_{\max}$ của VNS, độ dài chuỗi $\ell_{\max}$, số nghiệm khởi tạo và giới hạn thời gian đều ảnh hưởng trực tiếp đến chất lượng nghiệm. Việc chọn tham số chưa được khảo sát một cách hệ thống trên nhiều tổ hợp.

Thứ năm, đồ án mới tập trung vào phiên bản heuristic của bài toán, chưa so sánh trực tiếp với nghiệm tối ưu hoặc cận dưới từ mô hình exact. Vì vậy, gap được sử dụng trong thực nghiệm là gap so với nghiệm tốt nhất trong ba giải thuật, không phải gap tối ưu tuyệt đối.

7.4 Hướng phát triển

Từ các hạn chế trên, có thể phát triển đồ án theo một số hướng sau.

7.4.1 Cải tiến VNS

Kết quả thực nghiệm cho thấy VNS là hướng cần được cải tiến nhiều nhất. Một hướng trực tiếp là thiết kế lại cơ chế shaking để tạo thay đổi có định hướng hơn. Thay vì áp dụng destroy–repair lặp lại một cách tương đối đơn giản, shaking có thể ưu tiên các cạnh thuộc điểm triển khai nghẽn, các cạnh có chi phí chèn cao, hoặc các cạnh nằm xa cụm hiện tại của điểm triển khai.

Ngoài ra, có thể sử dụng quy tắc chấp nhận linh hoạt hơn. Phiên bản hiện tại chủ yếu chấp nhận cải thiện nghiệm ngặt. Trong một số trường hợp, cho phép chấp nhận nghiệm không tốt hơn trong ngắn hạn, chẳng hạn theo simulated annealing hoặc threshold accepting, có thể giúp thuật toán rời khỏi các vùng cực trị cục bộ sâu hơn.

Một hướng khác là dùng nhiều loại shaking thay vì một loại duy nhất. Ví dụ, shaking ở mức nhỏ có thể là dịch chuyển một số cạnh khỏi điểm nghẽn, shaking ở mức trung bình có thể là hoán đổi cụm cạnh giữa hai điểm triển khai, còn shaking ở mức lớn có thể là tái chọn một phần điểm triển khai. Cách này phù hợp hơn với tinh thần thay đổi lân cận của VNS.

7.4.2 Phát triển LNS thích nghi

LNS cho kết quả tốt nhất trong thực nghiệm, vì vậy một hướng phát triển tự nhiên là mở rộng thành Adaptive Large Neighborhood Search (ALNS). Trong ALNS, nhiều toán tử destroy và repair được sử dụng song song, và trọng số chọn toán tử được cập nhật dựa trên hiệu quả trong quá trình chạy.

Các toán tử destroy có thể bao gồm:

- phá ngẫu nhiên để tạo đa dạng;
- phá tại điểm triển khai ngắn để giảm trực tiếp makespan;
- phá các cạnh có chi phí chèn lớn;
- phá các cạnh gần nhau về mặt hình học để tái cấu trúc một vùng cục bộ;
- phá theo chuyển bay, tức loại bỏ toàn bộ một số chuyển bay kém hiệu quả.

Các toán tử repair có thể bao gồm chèn tốt nhất theo mục tiêu min-max, chèn gần nhất theo hình học, chèn có hối tiếc và chèn ưu tiên cân bằng tải. Việc kết hợp nhiều toán tử có thể giúp LNS ổn định hơn trên các thể hiện có cấu trúc khác nhau.

7.4.3 Chạy thực nghiệm thống kê rộng hơn

Một hướng cần thiết là mở rộng thực nghiệm theo hướng thống kê. Mỗi cấu hình nên được chạy với nhiều seed khác nhau. Khi đó, kết quả có thể được báo cáo bằng trung bình, độ lệch chuẩn, khoảng tin cậy và số lần đạt nghiệm tốt nhất. Điều này giúp đánh giá độ ổn định của từng giải thuật, đặc biệt với các phương pháp có thành phần ngẫu nhiên như VNS và LNS.

Ngoài hai nhóm MTLARP4 và MTLARP5, có thể bổ sung thêm các nhóm thể hiện lớn hơn hoặc các thể hiện có cấu trúc khác nhau về số điểm triển khai, số cạnh yêu cầu, độ phân tán hình học và giới hạn bay L . Khi đó, có thể phân tích rõ hơn giải thuật nào phù hợp với từng loại thể hiện.

7.4.4 Kết hợp với cận dưới và mô hình exact

Trong thực nghiệm hiện tại, chất lượng nghiệm được đánh giá tương đối giữa ba giải thuật. Để đánh giá chính xác hơn, cần xây dựng hoặc khai thác các cận dưới cho bài toán. Các cận dưới có thể đến từ mô hình quy hoạch nguyên nổi lỏng, từ bài toán phân công đơn giản hơn, hoặc từ các ràng buộc cân bằng tải.

Với các thể hiện nhỏ, có thể chạy mô hình exact hoặc branch-and-cut để lấy nghiệm tối ưu hoặc cận tốt. Khi đó, các giải thuật heuristic có thể được đánh giá bằng optimality gap thay vì chỉ so sánh nội bộ. Điều này giúp kết luận về chất lượng nghiệm có cơ sở mạnh hơn.

7.4.5 Cải tiến pha rời rạc hóa và chia nhỏ cạnh

Một đặc điểm quan trọng của MM-MT-dLARP là bài toán gốc có tính liên tục. Chất lượng nghiệm phụ thuộc vào cách rời rạc hóa các đường cần phục vụ. Nếu rời rạc hóa quá thô, nghiệm có thể bỏ lỡ các điểm vào-ra tốt trên đường. Nếu rời rạc hóa quá mịn, số đỉnh và số cạnh bay chết tăng nhanh, làm thời gian chạy lớn.

Do đó, một hướng phát triển quan trọng là xây dựng pha chia nhỏ cạnh thích nghi hơn. Thay vì chia đều toàn bộ các cạnh, thuật toán có thể chỉ thêm điểm trung gian tại những cạnh xuất hiện trong nghiệm tốt, tại các vùng có chi phí bay chết lớn, hoặc tại các đoạn nằm gần nhiều điểm triển khai tiềm năng. Sau mỗi lần chia nhỏ, nghiệm cũ được ánh xạ sang đồ thị mới và tiếp tục cải thiện bằng LNS hoặc VND.

7.4.6 Tối ưu hiệu năng cài đặt

Các toán tử chèn, dịch chuyển và hoán đổi thường phải xét nhiều vị trí trong nhiều chuyến bay. Khi kích thước thể hiện tăng, thời gian tính toán tăng nhanh. Vì vậy, cần tối ưu hiệu năng cài đặt bằng các kỹ thuật như lưu cache chi phí giữa các đỉnh, cập nhật delta cost thay vì tính lại toàn bộ nghiệm, giới hạn danh sách ứng viên chèn theo khoảng cách gần, và song song hóa các phép thử độc lập.

Đặc biệt, trong LNS, bước repair là phần tốn thời gian nhất. Nếu có thể giảm số vị trí chèn cần xét mà vẫn giữ chất lượng nghiệm, thuật toán có thể xử lý các thể hiện lớn hơn hoặc chạy nhiều seed hơn trong cùng thời gian.

7.5 Kết luận cuối cùng

MM-MT-dLARP là một bài toán có ý nghĩa thực tiễn trong các ứng dụng kiểm tra hạ tầng bằng drone, đồng thời có độ khó cao do kết hợp định tuyến trên cạnh, lựa chọn vị trí và cân bằng tải. Đồ án đã xây dựng được một khung biểu diễn nghiệm và các toán tử tìm kiếm cục bộ phù hợp với cấu trúc của bài toán.

Kết quả thực nghiệm cho thấy LNS là hướng tiếp cận hiệu quả nhất trong phạm vi nghiên cứu, nhờ khả năng tái cấu trúc nghiệm trên lân cận lớn và cân bằng lại tải giữa các điểm triển khai. VND tuy đơn giản hơn nhưng cho thời gian chạy tốt và có thể đóng vai trò quan trọng như một thủ tục tăng cường. VNS cần được cải tiến thêm để cơ chế shaking tạo ra các thay đổi có ích hơn.

Nhìn chung, các kết quả thu được xác nhận rằng nhóm phương pháp tìm kiếm cục bộ và metaheuristic là hướng tiếp cận phù hợp cho MM-MT-dLARP. Với các cải tiến về ALNS, thực nghiệm thống kê, cận dưới và rời rạc hóa thích nghi, hướng nghiên cứu này còn nhiều tiềm năng để phát triển thành một bộ giải hiệu quả hơn cho các bài toán định tuyến drone nhiều chuyến trong thực tế.

Tài liệu tham khảo

- [1] James F. Campbell, 'Angel Corber'an, Isaac Plana, and Jos'e Mar'ia Sanchis. Drone arc routing problems. *Networks*, 72(4):543–559, 2018.
- [2] James F. Campbell, 'Angel Corber'an, Isaac Plana, Jos'e Mar'ia Sanchis, and Pau Segura. Solving the length-constrained K-drones rural postman problem. *European Journal of Operational Research*, 292(1):60–72, 2021.
- [3] James F. Campbell, 'Angel Corber'an, Isaac Plana, Jos'e Mar'ia Sanchis, and Pau Segura. Polyhedral analysis and a new algorithm for the length-constrained K-drones rural postman problem. *Computational Optimization and Applications*, 83:67–109, 2022.
- [4] Sung Hoon Chung, Bipan Sah, and Jihye Lee. Optimization for drone and drone-truck combined operations: A review of the state of the art and future directions. *Computers & Operations Research*, 123:105004, 2020.
- [5] Teresa Corber'an, Isaac Plana, and Jos'e Mar'ia Sanchis. The min max multi-trip drone location arc routing problem. *Computers & Operations Research*, 174:106894, 2025.
- [6] Pierre Hansen and Nenad Mladenovi'c. Variable neighborhood search: Principles and applications. *European Journal of Operational Research*, 130(3):449–467, 2001.
- [7] Pierre Hansen, Nenad Mladenovi'c, and Jos'e A. Moreno P'erez. Variable neighbourhood search: Methods and applications. *Annals of Operations Research*, 175:367–407, 2010.
- [8] Giacomo Macrina, Luigi Di Puglia Pugliese, Francesca Guerriero, and Gilbert Laporte. Drone-aided routing: A literature review. *Transportation Research Part C: Emerging Technologies*, 120:102762, 2020.
- [9] Nenad Mladenovi'c and Pierre Hansen. Variable neighborhood search. *Computers & Operations Research*, 24(11):1097–1100, 1997.
- [10] David Pisinger and Stefan Ropke. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34(8):2403–2435, 2007.

-
- [11] Stefan Ropke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006.
- [12] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming – CP 1998*, volume 1520 of *Lecture Notes in Computer Science*, pages 417–431. Springer, 1998.